ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CSC17001 – Intelligent Data Analytics

EXPLORING MENTAL HEALTH DATA

Dự đoán trạng thái trầm cảm từ dữ liệu khảo sát sức khỏe tinh thần

Giảng viên: Nguyễn Tiến Huy, Nguyễn Thanh Tình, Lê Thanh Tùng
Lớp học phần: Phân tích dữ liệu thông minh
Nhóm: Hít Khí Trời Làm Data

TP. Hồ Chí Minh, Ngày 25 tháng 4 năm 2026

Mục lục

1 Giới thiệu dự án **1**
1.1 Bối cảnh bài toán . . . 1
1.2 Mục tiêu của đồ án . . . 1
1.3 Phạm vi thực hiện . . . 1
1.4 Phân công công việc . . . 1

2 Tổng quan bài toán và cơ sở phương pháp **2**
2.1 Phát biểu bài toán . . . 2
2.2 Đặc điểm của bài toán phân loại sức khỏe tinh thần . . . 2
2.3 Các độ đo đánh giá . . . 2
2.4 Nhóm mô hình nên khảo sát . . . 3
2.5 Chiến lược đánh giá thực nghiệm . . . 3

3 Khám phá dữ liệu **3**
3.1 Mô tả bộ dữ liệu . . . 3
3.2 Kiểm tra chất lượng dữ liệu . . . 5
3.3 Các trực quan hóa nên đưa vào báo cáo . . . 7
3.4 Kiểm định Chi-squared cho các biến phân loại . . . 11
3.5 Nhận xét rút ra từ EDA . . . 12
3.6 Liên hệ với các câu hỏi nghiên cứu và insight tiềm ẩn . . . 13
3.6.1 Silent Depression: vì sao không thể chỉ dựa vào "báo động đỏ" lâm sàng . . . 13
3.6.2 Áp lực và giấc ngủ quan trọng hơn điểm số tuyệt đối . . . 14
3.6.3 City effect và ý nghĩa thực tiễn của các biến vĩ mô . . . 16

4 Tiền xử lý dữ liệu và tạo đặc trưng **16**
4.1 Làm sạch dữ liệu . . . 16
4.2 Xử lý giá trị thiếu . . . 17
4.3 Mã hóa biến và chuẩn hóa thang đo . . . 17
4.4 Xử lý mất cân bằng lớp . . . 18
4.5 Feature engineering / feature selection . . . 19

5 Phát triển mô hình **21**
5.1 Mô hình baseline . . . 21
5.1.1 Logistic Regression – baseline tuyến tính dễ diễn giải . . . 21
5.1.2 Random Forest – baseline phi tuyến trên dữ liệu dạng cây . . . 21
5.1.3 Diễn giải chung cho giai đoạn baseline . . . 22
5.2 Gradient Boosting Models . . . 22
5.2.1 Tối ưu siêu tham số với Optuna . . . 23
5.2.2 XGBoost . . . 23
5.2.3 LightGBM . . . 23
5.2.4 CatBoost . . . 24
5.3 OOF Predictions cho Blending và Stacking . . . 24
5.4 Blending Ensemble . . . 25
5.5 Neural Network (MLP) . . . 25
5.5.1 Kiến trúc và cấu hình huấn luyện . . . 26

5.5.2 Kết quả và vai trò trong hệ thống 26
5.6 Kết luận của giai đoạn mô hình hóa 26
5.6.1 Bảng tổng kết kết quả mô hình 27

6 Model Insights: Feature Importance, ROC, Confusion Matrix & Summary 27
6.1 Feature Importance – Tín hiệu nào chi phối mô hình? 27
6.2 ROC Curve – Khả năng phân biệt hai lớp 28
6.3 Confusion Matrix – Sai số nào đáng lo nhất? 29
6.4 Bảng tổng hợp 30

7 Blending, stacking ensemble và mô hình cuối cùng 30
7.1 Từ blending đến stacking ensemble 30
7.2 Vì sao blending và stacking đều hợp lý? 31
7.3 Diễn giải kết quả cuối cùng 31

8 SHAP Analysis: Giải thích dự đoán mô hình 32
8.1 Tại sao cần SHAP? 32
8.2 SHAP Beeswarm Plot – Cách đọc 32
8.3 SHAP Bar Plot – Cách đọc 33
8.4 Top 5 SHAP Features – Ý nghĩa lâm sàng 34

9 Kết luận và hướng phát triển 35
9.1 Kết luận 35
9.2 Hạn chế 35
9.3 Hướng phát triển 36
9.3.1 Ưu tiên cao 36
9.3.2 Ưu tiên trung bình 37
9.3.3 Ưu tiên thấp 37

1 Giới thiệu dự án

1.1 Bối cảnh bài toán

Bài toán *Exploring Mental Health Data* được xây dựng từ cuộc thi Kaggle cùng tên, với mục tiêu phân tích dữ liệu khảo sát sức khỏe tinh thần để xác định các yếu tố liên quan đến `Depression`. Đây là một bài toán **phân loại nhị phân**, trong đó mô hình dự đoán một cá nhân có biểu hiện trầm cảm hay không dựa trên các thuộc tính nhân khẩu học, học tập, nghề nghiệp, lối sống và tiền sử tâm lý.

1.2 Mục tiêu của đồ án

Báo cáo cần trả lời các câu hỏi chính sau:

- Dữ liệu khảo sát có những nhóm biến nào, chất lượng dữ liệu ra sao, và tồn tại những tín hiệu đáng chú ý nào?
- Những bước tiền xử lý và tạo đặc trưng nào là cần thiết để huấn luyện mô hình đáng tin cậy?
- Những mô hình phân loại nào phù hợp cho bài toán này và hiệu năng của chúng khác nhau như thế nào?
- Những đặc trưng nào có ảnh hưởng lớn nhất đến dự đoán `Depression`?

1.3 Phạm vi thực hiện

Nhóm tập trung vào bốn hạng mục chính: (1) tìm hiểu dữ liệu, (2) tiền xử lý và xây dựng đặc trưng, (3) huấn luyện và so sánh mô hình, và (4) phân tích kết quả để rút ra nhận xét. Nếu cuộc thi cung cấp cả `train.csv` và `test.csv`, toàn bộ quá trình chọn mô hình và điều chỉnh siêu tham số chỉ nên thực hiện trên tập huấn luyện nhằm tránh rò rỉ dữ liệu.

1.4 Phân công công việc

Nên bổ sung một bảng phân công công việc ngay đầu báo cáo để đáp ứng đúng rubric của môn học.

Thành viên	**Nhiệm vụ chính**	**Tỷ lệ hoàn thành**
Nguyễn Kim Quốc	Tiền xử lý dữ liệu & trích xuất đặc trưng	100%
Mai Đình Trí	Huấn luyện mô hình	[100%]
Nguyễn Đức Tiến	EDA + Huấn luyện mô hình	[100%]
Đào Quốc Tuấn	Huấn luyện mô hình	[100%]
Nguyễn Lê Tân Tiến	EDA + Tiền xử lý	[100%]
Kiều Đỗ Song Tinh	EDA + Report + Video	[100%]

Bảng 1: Bảng phân công công việc của nhóm.

Lưu ý

Tổng các tỷ lệ đóng góp nên được chuẩn hóa sao cho phản ánh đúng mức độ tham gia thực tế của từng thành viên. Nếu nhóm có 5–6 người, chỉ cần mở rộng bảng theo cùng cấu trúc.

2 Tổng quan bài toán và cơ sở phương pháp

2.1 Phát biểu bài toán

Gọi $\mathbf{x} \in \mathbb{R}^d$ là vector đặc trưng của một người tham gia khảo sát và $y \in \{0, 1\}$ là nhãn tương ứng, trong đó:

$$y = \begin{cases} 1 & \text{nếu cá nhân được gán là có dấu hiệu trầm cảm,} \\ 0 & \text{ngược lại.} \end{cases}$$

Nhiệm vụ của mô hình là học một hàm $f(\mathbf{x}) \approx P(y = 1 \mid \mathbf{x})$ để phục vụ dự đoán và phân tích các yếu tố liên quan.

2.2 Đặc điểm của bài toán phân loại sức khỏe tinh thần

Bài toán này thường có một số thách thức đặc trưng:

- **Dữ liệu hỗn hợp:** đồng thời chứa biến số, biến phân loại, biến thứ bậc và câu trả lời khảo sát.
- **Thiếu dữ liệu:** người tham gia có thể bỏ sót một số câu hỏi nhạy cảm.
- **Mất cân bằng lớp:** tỷ lệ mẫu thuộc lớp `Depression = 1` có thể thấp hơn đáng kể so với lớp còn lại.
- **Khả năng diễn giải:** ngoài độ chính xác, báo cáo cần lý giải được mô hình dựa trên tín hiệu nào để dự đoán.

2.3 Các độ đo đánh giá

Do đây là bài toán phân loại, nhóm nên báo cáo nhiều thước đo thay vì chỉ dùng Accuracy:

- **Accuracy:** tỷ lệ dự đoán đúng trên toàn bộ mẫu.
- **Precision:** mức độ chính xác của các mẫu được dự đoán là dương tính.
- **Recall:** khả năng phát hiện đầy đủ các trường hợp dương tính.
- **F1-score:** trung bình điều hòa của Precision và Recall.
- **ROC-AUC:** khả năng phân tách hai lớp trên nhiều ngưỡng khác nhau.

Nếu dữ liệu bị lệch lớp mạnh, nên ưu tiên phân tích thêm **PR-AUC**, **balanced accuracy**, hoặc **macro-F1**.

2.4 Nhóm mô hình nên khảo sát

Một cấu hình báo cáo hợp lý là chia mô hình thành ba tầng:

1. **Mô hình baseline:** Logistic Regression, Random Forest.
2. **Mô hình ensemble:** Random Forest, XGBoost/LightGBM/CatBoost nếu được phép dùng.
3. **Neuron Network**: Thiết lập các lớp MLP
4. **Mô hình có khả năng diễn giải:** Logistic Regression với hệ số, hoặc tree-based model kết hợp feature importance / SHAP.

2.5 Chiến lược đánh giá thực nghiệm

Nhóm nên mô tả rõ quy trình thực nghiệm như sau:

- Tách dữ liệu train/validation hoặc dùng cross-validation.
- Gói toàn bộ bước tiền xử lý và mô hình vào cùng một pipeline.
- Chỉ tuning siêu tham số trên tập train/validation.
- Báo cáo kết quả cuối cùng trên tập kiểm tra nội bộ hoặc tập hold-out.

3 Khám phá dữ liệu

3.1 Mô tả bộ dữ liệu

Phần này trình bày quy mô, cấu trúc và các đặc điểm nổi bật của bộ dữ liệu *Exploring Mental Health Data*. Tập dữ liệu được chia thành hai phần riêng biệt gồm **140,700 bản ghi** cho tập huấn luyện và **93,800 bản ghi** cho tập kiểm tra. Tổng thể, bộ dữ liệu có **20 cột**, trong đó bao gồm cột định danh, cột mục tiêu và **18 biến độc lập**. Quy mô này đủ lớn để thử nghiệm cả các mô hình học máy truyền thống lẫn các phương pháp ensemble hoặc deep learning.

Để thuận tiện cho việc phân tích, nhóm chia phần mô tả dữ liệu thành ba ý chính sau:

a) **Quy mô và cấu trúc dữ liệu.**

Bộ dữ liệu gồm hai bảng `train` và `test` với số lượng mẫu lần lượt là 140,700 và 93,800. Về kiểu dữ liệu, tập thuộc tính bao gồm **10 biến số** và **10 biến phân loại**. Các biến số đại diện cho những đại lượng định lượng như `Age`, `CGPA`, `Work/Study Hours` hay các thang đo như `Academic Pressure`, `Work Pressure`, `Study Satisfaction`, `Job Satisfaction`, và `Financial Stress`. Trong khi đó, các biến phân loại mô tả thông tin định danh hoặc hành vi như `Gender`, `City`, `Profession`, `Degree`, `Sleep Duration`, và `Dietary Habits`.

b) **Các nhóm biến chính.**

Dựa trên ngữ nghĩa, các thuộc tính có thể được chia thành bốn nhóm nội dung rõ rệt. Nhóm *thông tin cá nhân* gồm `Name`, `Gender`, `Age`, `City`. Nhóm *trạng thái nghề nghiệp/học thuật* gồm `Working Professional or Student`, `Profession`, `Degree`,

`CGPA`. Nhóm *chỉ số áp lực và hài lòng* gồm `Academic Pressure`, `Work Pressure`, `Study Satisfaction`, `Job Satisfaction`, `Financial Stress`. Cuối cùng, nhóm *thói quen và tiền sử* gồm `Sleep Duration`, `Dietary Habits`, `Work/Study Hours`, `Family History of Mental Illness`, và `Have you ever had suicidal thoughts?`. Biến mục tiêu là `Depression`, được mã hóa nhị phân với hai giá trị 0 và 1.

c) **Đặc tính phân phối của các biến số.**

Để đánh giá sơ bộ hình dạng phân phối của các biến định lượng, nhóm sử dụng các thống kê mô tả gồm trung bình, trung vị, độ lệch chuẩn, độ lệch (Skewness) và độ nhọn (Kurtosis). Kết quả cho thấy phần lớn các biến số có phân phối khá cân bằng quanh trung tâm và không xuất hiện hiện tượng lệch mạnh.

Biến	**Mean**	**Median**	**Std**	**Skewness**	**Kurtosis**
Age	40.388621	42.00	12.384099	-0.217977	-1.149302
Academic Pressure	3.142273	3.00	1.380457	-0.133977	-1.163739
Work Pressure	2.998998	3.00	1.405771	0.018515	-1.295189
CGPA	7.658636	7.77	1.464466	-0.073636	-1.229219
Study Satisfaction	2.944940	3.00	1.360197	0.011764	-1.225619
Job Satisfaction	2.974404	3.00	1.416078	0.054361	-1.304939
Work/Study Hours	6.252679	6.00	3.853615	-0.128169	-1.283424
Financial Stress	2.988983	3.00	1.413633	0.035603	-1.313662
Depression	0.181713	0.00	0.385609	1.650852	0.725321

Bảng 2: Thống kê mô tả và hình dạng phân phối của các biến số chính.

Từ bảng thống kê trên, nhóm rút ra ba nhận xét chính:

- **Thứ nhất, về tính đối xứng:** các chỉ số áp lực và hài lòng có giá trị Skewness gần bằng 0, cho thấy dữ liệu không bị thiên lệch rõ rệt về một phía và quy trình thu thập dữ liệu không nghiêng mạnh về một cực đánh giá.
- **Thứ hai, về độ tập trung:** các giá trị Kurtosis âm xuất hiện khá đồng đều ở nhiều biến cho thấy phân phối có xu hướng bẹt hơn chuẩn, đồng nghĩa với việc dữ liệu bao phủ nhiều kịch bản khác nhau của đối tượng khảo sát.
- **Thứ ba, đối với biến mục tiêu:** giá trị trung bình của `Depression` là 0.181713, tương ứng khoảng **18.17%** mẫu dương tính; đây là tín hiệu rõ ràng cho thấy bài toán có hiện tượng mất cân bằng lớp và cần được lưu ý ở giai đoạn xây dựng mô hình.

Bảng 3: Bảng định nghĩa các cột trong bộ dữ liệu khảo sát sức khỏe tinh thần.

Cột	**Phân loại**	**Ý nghĩa / Định nghĩa**
`id`	Identifier	ID người tham gia.
`Name`	Identifier	Tên người dùng.
`Gender`	Categorical	Giới tính.

Cột	Phân loại	Ý nghĩa / Định nghĩa
`Age`	Numerical	Độ tuổi.
`City`	Categorical	Nơi cư trú.
`Working Professional or Student`	Categorical	Tình trạng lao động hoặc học tập.
`Profession`	Categorical	Ngành nghề hoặc môn học.
`Academic Pressure`	Numerical	Chỉ số áp lực học tập.
`Work Pressure`	Numerical	Chỉ số áp lực công việc.
`CGPA`	Numerical	Điểm hệ số.
`Study Satisfaction`	Numerical	Điểm hài lòng học hành.
`Job Satisfaction`	Numerical	Điểm hài lòng công việc.
`Sleep Duration`	Ordinal Categorical	Thói quen ngủ.
`Dietary Habits`	Ordinal Categorical	Thói quen ăn uống.
`Degree`	Categorical	Bằng cấp cao nhất.
`Have you ever had suicidal thoughts ?`	Categorical (Binary)	Từng suy nghĩ tự tử hay chưa.
`Work/Study Hours`	Numerical	Số giờ làm việc hoặc học mỗi ngày.
`Financial Stress`	Numerical	Điểm số căng thẳng tài chính.
`Family History of Mental Illness`	Categorical (Binary)	Có tiền sử bệnh tâm lý gia đình hay không.
`Depression`	Target (Binary)	Trầm cảm, là biến mục tiêu dự đoán.

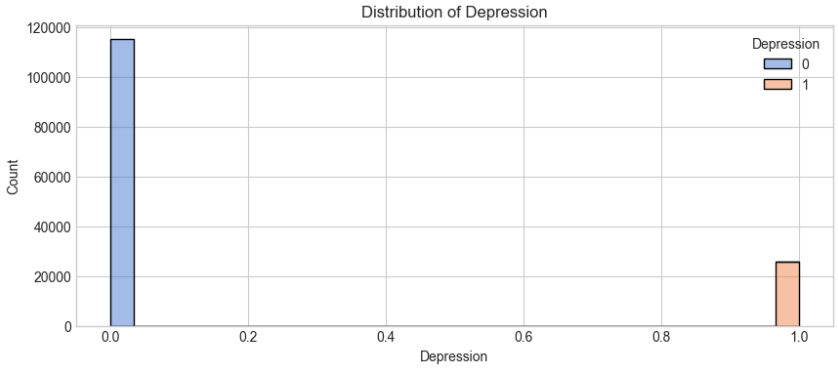
3.2 Kiểm tra chất lượng dữ liệu

Trong bước kiểm tra chất lượng dữ liệu, nhóm tập trung vào tỷ lệ giá trị thiếu của từng cột, số lượng bản ghi trùng lặp, phân bố nhãn `Depression`, phân bố của các biến số quan trọng và tần suất xuất hiện của các nhóm giá trị trong biến phân loại.

- **Tình trạng dữ liệu bị thiếu (Missing Values).** Có một cấu trúc khuyết thiếu dữ liệu mang tính logic rất rõ ràng, phụ thuộc rẽ nhánh vào cột `Working Professional or Student`:
  - **Nhóm sinh viên (Students):** Các cột liên quan đến học thuật như `Academic Pressure` (khoảng **27,897** giá trị không rỗng), `CGPA` (khoảng **27,898** giá trị không rỗng), và `Study Satisfaction` (khoảng **27,897** giá trị không rỗng) có số lượng bản ghi xấp xỉ nhau, vào khoảng 28 nghìn mẫu. Điều này phù hợp với quy mô của nhóm sinh viên trong bộ dữ liệu. Với những người đã đi làm, các cột này bị bỏ trống dưới dạng `NaN`.
  - **Nhóm người đi làm (Working Professionals):** Các cột liên quan đến công việc như `Profession` (khoảng **104,070** giá trị không rỗng), `Work Pressure` (khoảng **112,782** giá trị không rỗng), và `Job Satisfaction` (khoảng **112,790** giá trị không rỗng) có lượng dữ liệu phù hợp với nhóm người đi làm. Ngược lại, sinh viên sẽ bị khuyết các giá trị này.
  - **Khuyết thiếu ngẫu nhiên / thiểu số:** Một số cột như `Dietary Habits` (thiếu 4 dòng), `Degree` (thiếu 2 dòng), và `Financial Stress` (thiếu 4 dòng) có số lượng giá trị thiếu rất nhỏ, gần như không đáng kể so với toàn bộ tập dữ liệu.

- **Số lượng giá trị trùng lặp:** Kết quả kiểm tra cho thấy **không có bản ghi nào bị trùng lặp hoàn toàn** trên toàn bộ tập dữ liệu. Đồng thời, ngay cả khi bỏ qua cột `id`, số lượng bản ghi trùng lặp vẫn bằng **0**. Điều này cho thấy dữ liệu không chứa các mẫu lặp rõ ràng có thể gây sai lệch cho thống kê mô tả hoặc quá trình huấn luyện mô hình.

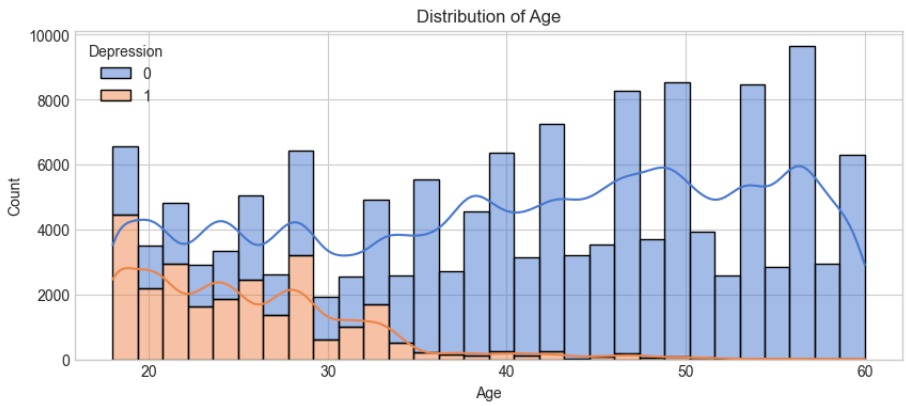
- **Phân bố nhãn `Depression`:** Biểu đồ phân bố nhãn ở Hình 1 cho thấy lớp `0` chiếm đa số rõ rệt, trong khi lớp `1` chỉ chiếm khoảng **18.17%**. Điều này xác nhận bài toán có hiện tượng *class imbalance*, vì vậy ở giai đoạn huấn luyện cần ưu tiên các thước đo như F1-score, Recall hoặc AUC thay vì chỉ nhìn vào Accuracy.



Hình 1: Biểu đồ phân bố nhãn mục tiêu `Depression`.

Nhìn chung, biểu đồ cho thấy số lượng mẫu không trầm cảm lớn hơn rất nhiều so với nhóm có trầm cảm. Sự chênh lệch này có thể khiến mô hình thiên về dự đoán lớp đa số nếu không có chiến lược xử lý phù hợp như cân bằng dữ liệu, gán trọng số lớp hoặc chọn ngưỡng phân loại hợp lý.

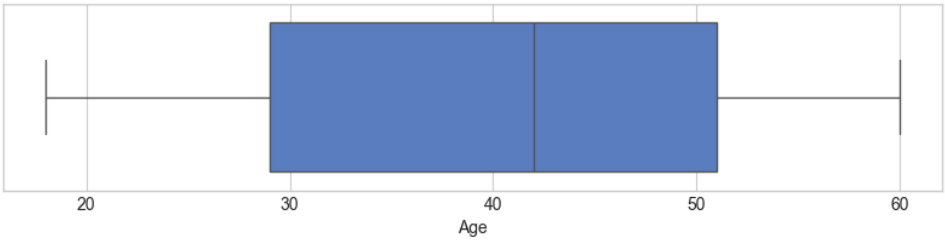
- **Phân bố của các biến số quan trọng:** Hình 2 minh họa phân bố của một số biến quan trọng trong tập dữ liệu. Quan sát trực quan cho thấy phần lớn biến có phân bố trải khá rộng, phản ánh sự đa dạng giữa các đối tượng khảo sát; đồng thời vẫn có những vùng tập trung mật độ cao cần được lưu ý khi chuẩn hóa hoặc xử lý ngoại lai.



Hình 2: Biểu đồ phân bố của một số biến số quan trọng trong bộ dữ liệu.

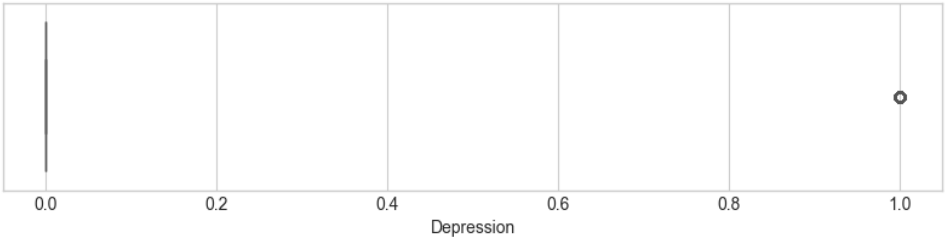
Nhìn chung, các biểu đồ này giúp nhận diện nhanh hình dạng phân phối thực tế của dữ liệu, bao gồm mức độ lệch, độ phân tán và khả năng xuất hiện các giá trị ngoại lai. Đây là cơ sở quan trọng để lựa chọn các bước tiền xử lý phù hợp như chuẩn hóa, biến đổi thang đo hoặc cắt ngưỡng ngoại lệ nếu cần.

- **Phân bố giá trị ngoại lai:** Hình 3 và Hình 4 hỗ trợ nhận diện mức độ xuất hiện của các giá trị ngoại lai trong dữ liệu. Đây là bước quan trọng để đánh giá xem có cần giới hạn ngưỡng, biến đổi dữ liệu hay giữ nguyên các giá trị cực trị nhằm bảo toàn tín hiệu dự đoán.



Hình 3: Biểu đồ ngoại lai của các biến số quan trọng.

Đối với các biến dự đoán quan trọng, kết quả kiểm tra bằng IQR Fence và Z-Score đều hầu như không ghi nhận ngoại lai đáng kể. Điều này phù hợp với bản chất của bộ dữ liệu tổng hợp, nơi các giá trị thường được ràng buộc trong các khoảng hợp lý và ít xuất hiện điểm cực đoan. Vì vậy, ở bước tiền xử lý, nhóm không cần ưu tiên các kỹ thuật lọc ngoại lai mạnh cho các biến số này.



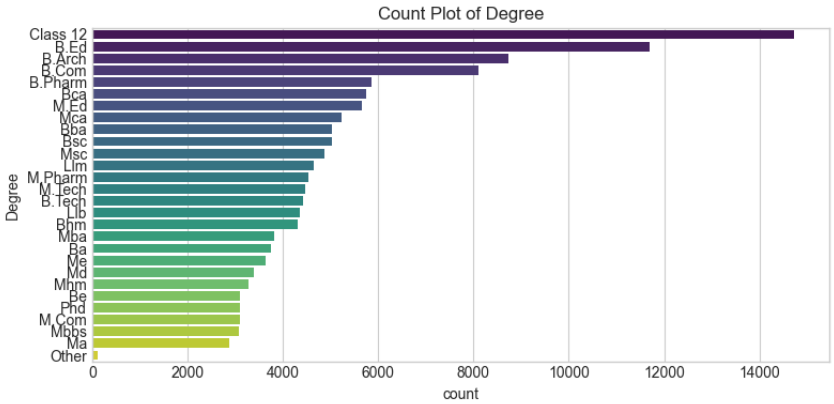
Hình 4: Biểu đồ ngoại lai theo nhãn `Depression`.

Riêng với biến mục tiêu `Depression`, IQR có thể gắn cờ nhiều "ngoại lệ" chỉ vì đây là biến nhị phân và phân bố lớp bị lệch mạnh về giá trị `0`. Khi đó, toàn bộ các mẫu `Depression = 1` dễ bị xem là vượt ngưỡng, dù thực chất đây không phải dữ liệu bất thường mà là lớp cần dự đoán. Do đó, phát hiện ngoại lệ không nên áp dụng máy móc cho biến mục tiêu hoặc các biến phân loại.

3.3 Các trực quan hóa nên đưa vào báo cáo

Để phần EDA thuyết phục và đúng yêu cầu "có hình minh họa", nhóm nên ưu tiên:

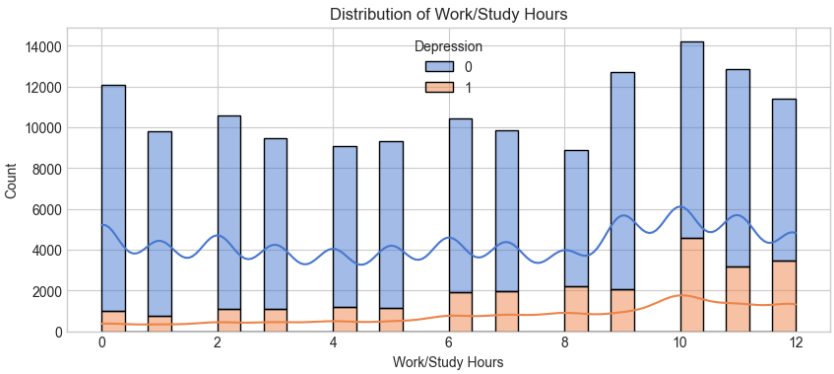
a) Countplot cho biến phân loại có nhiều nhóm như `Degree`;



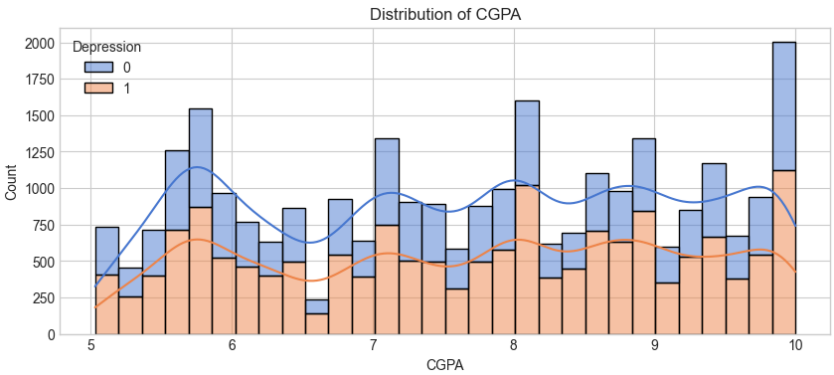
Hình 5: Biểu đồ tần suất của biến phân loại `Degree`.

Hình 5 cho thấy biến `Degree` có số lượng nhóm khá lớn và phân bố không đồng đều giữa các nhãn. Một vài nhóm như `Class 12`, `B.Ed` hoặc `B.Arch` xuất hiện với tần suất cao hơn rõ rệt, trong khi nhiều nhóm khác có tần suất thấp. Điều này cho thấy cần cân nhắc gom các nhóm hiếm hoặc áp dụng phương pháp mã hóa phù hợp để tránh làm dữ liệu bị phân mảnh quá mức trong bước tiền xử lý.

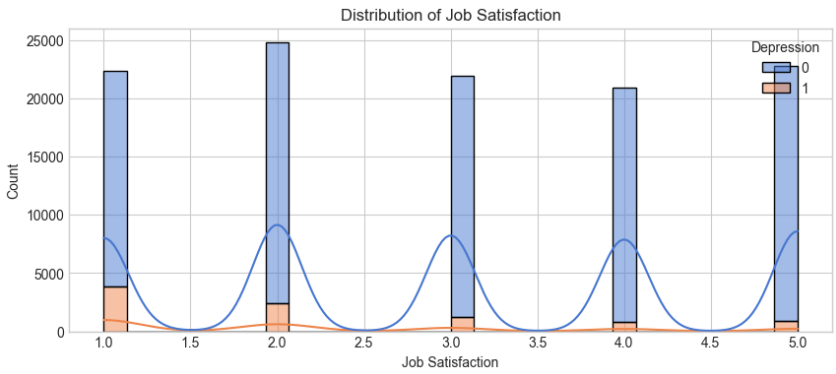
b) Histogram / KDE cho các biến số quan trọng như `Work/Study Hours`, `CGPA`, `Job Satisfaction` và `Study Satisfaction`;



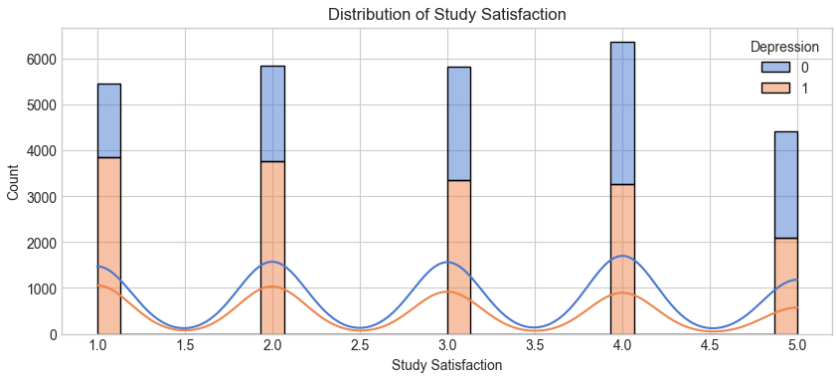
Hình 6: Phân bố của biến `Work/Study Hours` theo nhãn `Depression`.



Hình 7: Phân bố của biến `CGPA` theo nhãn `Depression`.



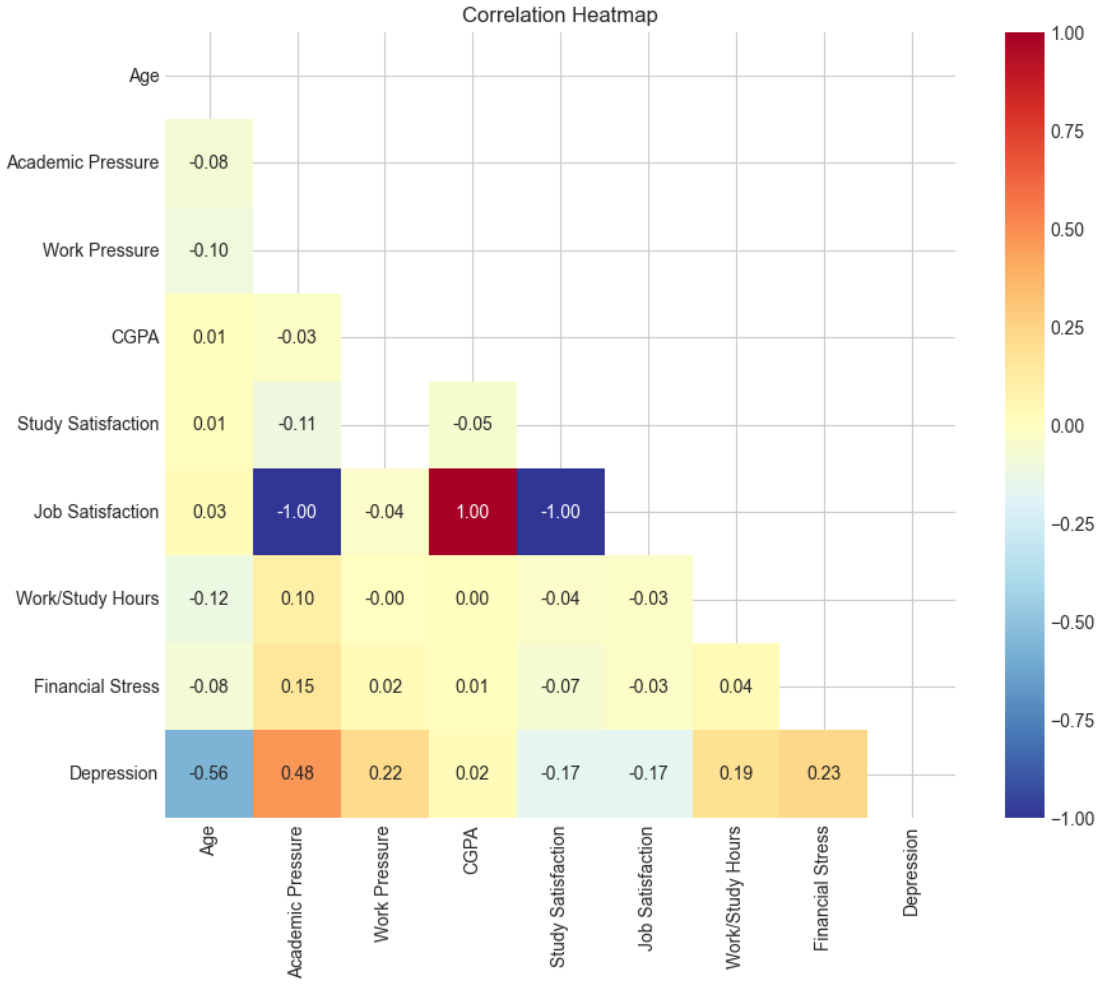
Hình 8: Phân bố của biến `Job Satisfaction` theo nhãn `Depression`.



Hình 9: Phân bố của biến `Study Satisfaction` theo nhãn `Depression`.

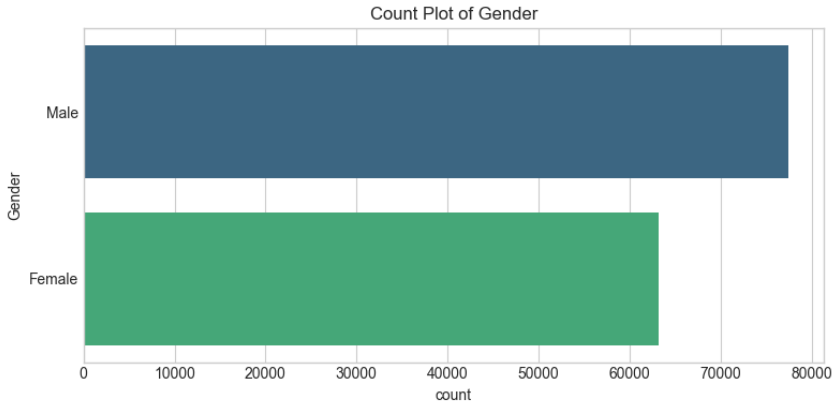
Các biểu đồ trên cho thấy một số biến có sự khác biệt phân bố tương đối rõ giữa hai nhóm `Depression`, đặc biệt ở số giờ học/làm việc và các thang đo mức độ hài lòng. Trong khi đó, `CGPA` có xu hướng phân bố khá dàn trải và chồng lấn nhiều hơn giữa hai lớp. Những khác biệt này gợi ý rằng các biến về cường độ học tập/công việc và mức độ hài lòng có thể mang nhiều tín hiệu hơn cho bài toán dự đoán trầm cảm.

c) Heatmap tương quan giữa các biến số;

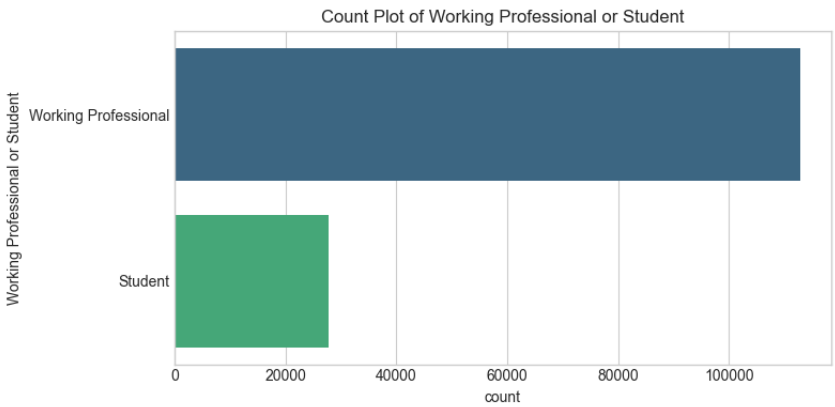


Hình 10: Hình minh họa bổ sung được sử dụng trong phần phân tích dữ liệu.

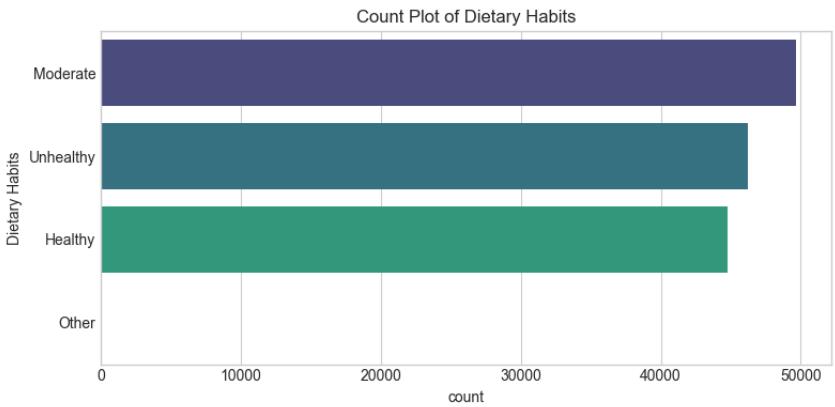
d) Countplot cho các biến phân loại nổi bật như giới tính, tình trạng học tập/làm việc và thói quen ăn uống.



Hình 11: Biểu đồ tần suất của biến `Gender`.



Hình 12: Biểu đồ tần suất của biến `Working Professional or Student`.



Hình 13: Biểu đồ tần suất của biến `Dietary Habits`.

Các countplot trên cho thấy một số biến phân loại có phân bố khá lệch giữa các nhóm. Biến `Gender` nghiêng nhẹ về nam, biến `Working Professional or Student` chủ yếu tập trung ở nhóm người đi làm, còn `Dietary Habits` phân bố chủ yếu quanh các mức `Moderate`, `Unhealthy` và `Healthy`. Những chênh lệch này cần được lưu ý trong bước mã hóa và phân tích đặc trưng để tránh bỏ sót các nhóm thiểu số.

3.4 Kiểm định Chi-squared cho các biến phân loại

Để đánh giá mức độ liên hệ giữa một số biến phân loại và biến mục tiêu `Depression`, nhóm sử dụng kiểm định *Chi-squared test of independence*. Ba chỉ số được báo cáo gồm: **Chi**^{2} để đo độ lệch giữa tần suất quan sát và tần suất kỳ vọng, **p-value** để kiểm tra ý nghĩa thống kê của giả thuyết độc lập, và **Cramér's V** để lượng hóa cường độ liên hệ thực tế giữa hai biến phân loại.

Biến	Chi^2	p-value	Cramér's V	Mức độ
`Gender`	9.29	0.0023	0.0081	Rất yếu
`Sleep Duration`	1197.45	≈ 0	0.0923	Yếu–trung bình
`Working Professional or Student`	38157.59	≈ 0	0.5208	Mạnh

Bảng 4: Tổng hợp kết quả kiểm định Chi-squared giữa các biến phân loại tiêu biểu và `Depression`.

- **Biến `Gender`:** tuy kiểm định cho kết quả có ý nghĩa thống kê ($p = 0.0023$), nhưng *Cramér's V* chỉ đạt **0.0081**, cho thấy mức liên hệ gần như không đáng kể về mặt thực tiễn. Tỷ lệ trầm cảm giữa nam và nữ gần như tương đương, vì vậy đây không phải là biến phân biệt mạnh cho mô hình.

- **Biến `Sleep Duration`:** có mối liên hệ rõ hơn với `Depression`, với $\textbf{Chi}^2 = \textbf{1197.45}$ và *Cramér's V* = **0.0923**. Kết quả cho thấy nhóm ngủ ít hơn `5` giờ có tỷ lệ trầm cảm cao nhất, trong khi nhóm ngủ trên `8` giờ có tỷ lệ thấp nhất. Đây là biến có giá trị dự báo và phù hợp với cách mã hóa thứ bậc.

- **Biến `Working Professional or Student`:** là biến nổi bật nhất trong nhóm kiểm định, với $\textbf{Chi}^2 = \textbf{38157.59}$ và *Cramér's V* = **0.5208**. Tỷ lệ trầm cảm ở nhóm sinh viên cao vượt trội so với nhóm người đi làm, cho thấy tình trạng học tập/làm việc là một đặc trưng rất mạnh cần được ưu tiên giữ lại trong bước xây dựng mô hình.

Nhìn chung, kiểm định Chi-squared cho thấy không phải mọi biến có ý nghĩa thống kê đều có giá trị phân biệt cao về mặt thực tiễn. Trong ba biến được kiểm tra, `Gender` gần như không đóng góp nhiều cho dự đoán, `Sleep Duration` mang tín hiệu ở mức vừa phải, còn `Working Professional or Student` là biến phân loại quan trọng nhất và nên được ưu tiên trong *feature engineering*.

3.5 Nhận xét rút ra từ EDA

a) **Tín hiệu dự báo quan trọng nhất nằm ở tuổi, căng thẳng tài chính và trạng thái học tập/làm việc.**

Phân tích cho thấy nhóm có `Depression = 1` tập trung nhiều hơn ở người trẻ, đặc biệt là sinh viên, đồng thời đi kèm mức `Financial Stress` cao hơn rõ rệt. Trong đó, biến `Working Professional or Student` là đặc trưng phân loại mạnh nhất theo kiểm định Chi-squared, nên cần được ưu tiên giữ lại và khai thác thêm ở bước *feature engineering*.

b) **Cấu trúc giá trị thiếu mang tính logic nghiệp vụ, không nên xử lý như thiếu ngẫu nhiên.**

Phần lớn giá trị khuyết ở các cột học thuật và công việc phản ánh sự phân nhánh tự nhiên giữa sinh viên và người đi làm, thay vì lỗi thu thập dữ liệu. Vì vậy, bước tiền xử lý cần tôn trọng ngữ nghĩa của từng nhóm, tránh nội suy bằng trung bình hoặc trung vị chung vì có thể làm mất cấu trúc thực của dữ liệu.

c) **Một số biến phân loại cần được làm sạch trước khi mã hóa.**

Các cột như `Sleep Duration` và `Dietary Habits` xuất hiện giá trị rác hoặc nhãn không nhất quán, có thể làm sai lệch phân bố nếu đưa thẳng vào mô hình. Do đó, cần chuẩn hóa nhãn, gom nhóm bất thường vào `Other/Unknown`, rồi mới áp dụng chiến lược mã hóa phù hợp như ordinal encoding cho biến có thứ bậc và binary encoding cho biến hai trạng thái.

d) **Bài toán có mất cân bằng lớp, nên cần ưu tiên thước đo đánh giá phù hợp.**

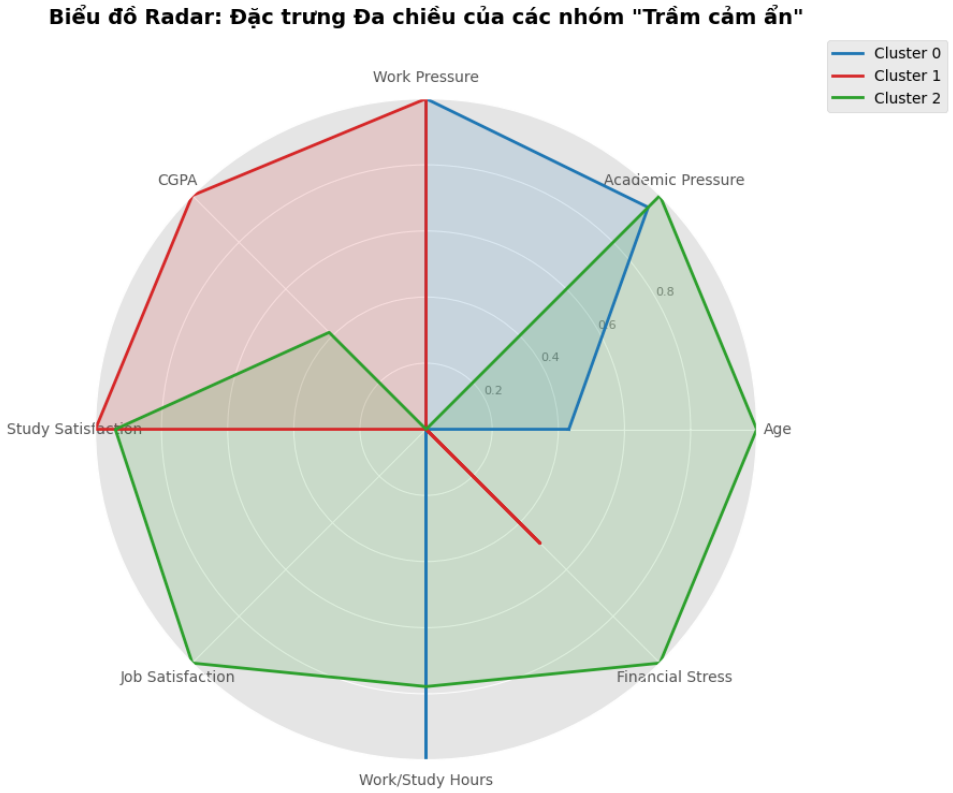
Tỷ lệ mẫu dương tính của `Depression` chỉ vào khoảng **18.17%**, cho thấy lớp dương bị thiểu số rõ rệt. Vì vậy, ở giai đoạn huấn luyện mô hình, nhóm nên ưu tiên các chỉ số như *F1-score*, *Recall* hoặc *AUC*, đồng thời cân nhắc gán trọng số lớp hoặc các kỹ thuật tái cân bằng dữ liệu nếu mô hình có xu hướng lệch về lớp đa số.

3.6 Liên hệ với các câu hỏi nghiên cứu và insight tiềm ẩn

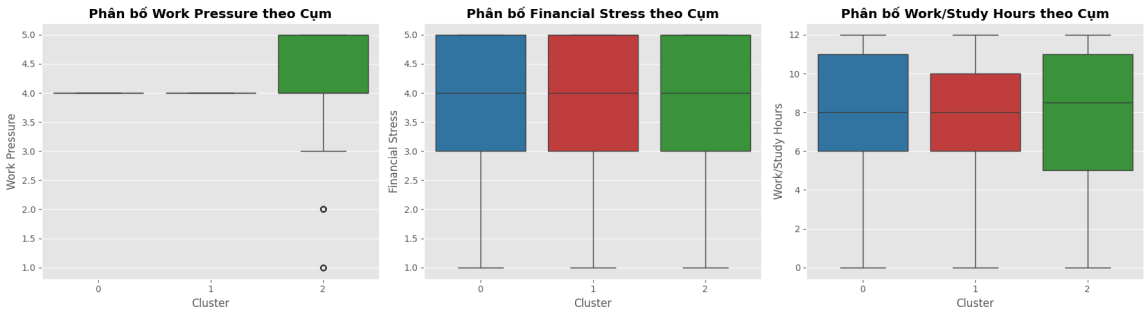
Một điểm mạnh của hệ thống không chỉ nằm ở việc đạt điểm `AUC` hay `F1-score` cao, mà còn ở chỗ các tín hiệu mô hình học được khá nhất quán với những phân tích chuyên sâu trong notebook `research_question`. Điều này giúp phần boosting không chỉ dừng ở mức *prediction*, mà còn có giá trị giải thích cho các mẫu nguy cơ tiềm ẩn trong dữ liệu.

3.6.1 Silent Depression: vì sao không thể chỉ dựa vào "báo động đỏ" lâm sàng

Phân tích câu hỏi nghiên cứu về *silent depression* cho thấy có **1,695** trường hợp thuộc nhóm `Depression = 1` nhưng đồng thời trả lời `No` với cả hai biến `Family History of Mental Illness` và `Have you ever had suicidal thoughts ?`. Nhóm này chiếm khoảng **6.6%** tổng số ca trầm cảm trong tập train. Kết quả này đặc biệt quan trọng đối với phần mô hình hóa, vì nó chứng minh rằng nếu hệ thống chỉ bám vào các "red flags" lâm sàng truyền thống thì sẽ bỏ sót một nhóm nguy cơ không nhỏ.
Các mô hình boosting vì thế có lợi thế rõ rệt so với cách sàng lọc thủ công đơn giản: chúng có thể kết hợp đồng thời tín hiệu từ **tuổi**, **áp lực học tập/công việc**, **mức độ hài lòng**, **stress tài chính**, **thói quen ngủ nghỉ** và các đặc trưng tổng hợp do nhóm thiết kế. Đây cũng là lý do các feature engineered như `Total_Stress_Sum`, `Satisfaction_Gap` hay các biến hành vi vẫn giữ được vai trò quan trọng ở các phân tích sau đó.



Hình 14: Biểu đồ radar mô tả đặc trưng đa chiều của ba cụm *silent depression*.



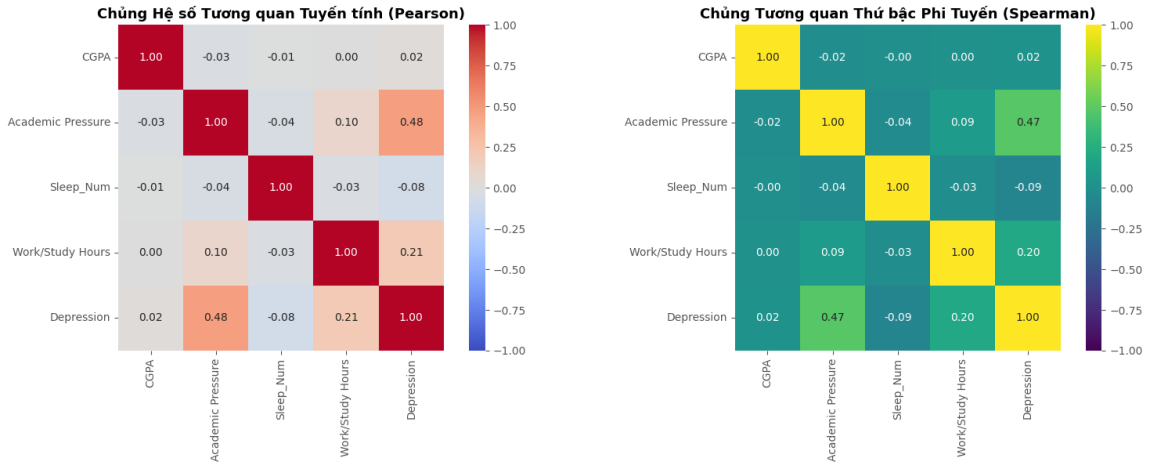
Hình 15: Boxplot so sánh `Work Pressure`, `Financial Stress` và `Work/Study Hours` giữa các cụm *silent depression*.

3.6.2 Áp lực và giấc ngủ quan trọng hơn điểm số tuyệt đối

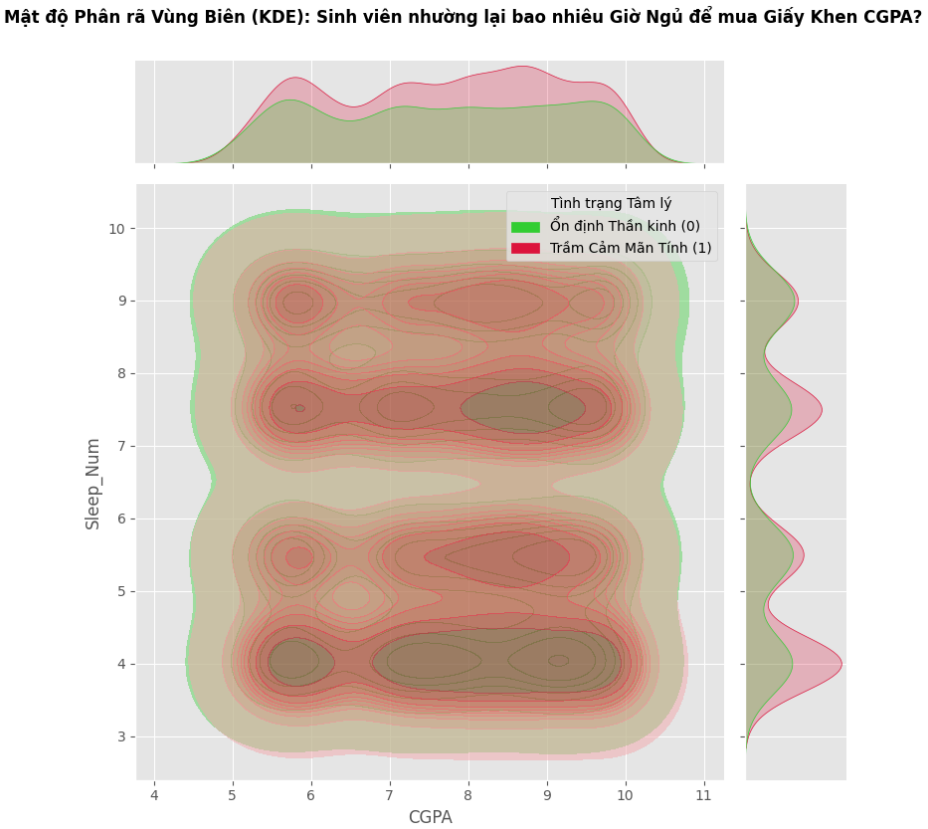
Một insight nổi bật khác từ notebook nghiên cứu là ở nhóm sinh viên, `CGPA` gần như không có tương quan đáng kể với `Depression` (xấp xỉ **0.02**), trong khi `Academic Pressure` lại có tương quan mạnh hơn nhiều (khoảng **0.48**). Nói cách khác, thành tích học thuật tự thân không phải là biến phân tách tốt; điều nguy hiểm hơn là cảm giác bị đè nén kéo dài bởi kỳ vọng và cường độ học tập.

Phân tích contour về giấc ngủ còn gợi ý một ranh giới thực tiễn rất đáng chú ý: nhóm trầm cảm tập trung dày hơn quanh mức ngủ khoảng **4–5 giờ**, trong khi vùng ngủ khoảng **7.5 giờ** xuất hiện nhiều hơn ở nhóm ổn định. Insight này giúp giải thích vì sao các mô hình boosting đánh giá cao các tín hiệu liên quan đến **stress**, **sự hài lòng** và **thói quen sinh hoạt**, thay vì chỉ dựa trên một biến thành tích như `CGPA`. Về mặt

báo cáo, đây là một luận điểm rất tốt để nhấn mạnh rằng mô hình đang học ra các mẫu nguy cơ tâm lý thực chất hơn là chỉ học lại các stereotype phổ biến.



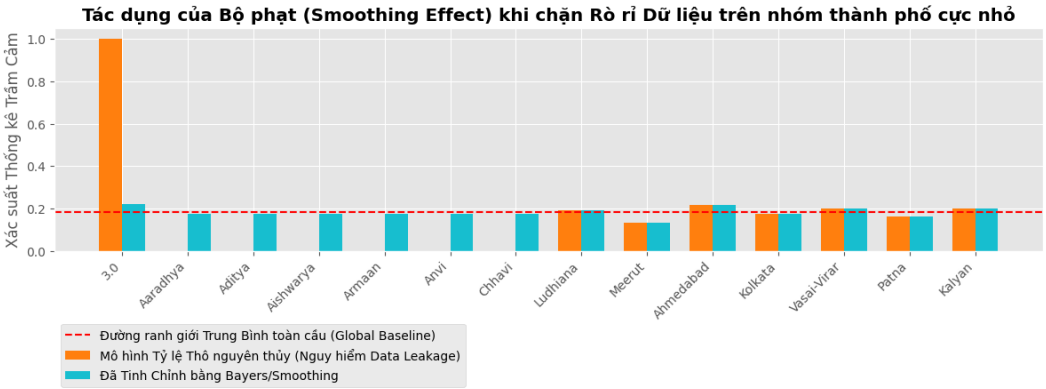
Hình 16: Ma trận tương quan Pearson và Spearman cho nhóm sinh viên: `Academic Pressure` nổi bật hơn `CGPA` trong liên hệ với `Depression`.



Hình 17: Biểu đồ KDE cho thấy mật độ nhóm trầm cảm dày hơn quanh vùng ngủ 4–5 giờ, trong khi vùng 7.5 giờ nghiêng nhiều hơn về nhóm ổn định.

3.6.3 City effect và ý nghĩa thực tiễn của các biến vĩ mô

Ở câu hỏi nghiên cứu về "hiệu ứng thành phố", mô hình hồi quy logistic đa biến cho thấy biến `City_Encoded` vẫn có ý nghĩa thống kê rất mạnh sau khi đã kiểm soát các yếu tố cá nhân, với **Odds Ratio = 1.11**. Dù độ lớn tác động không quá mạnh như nhóm áp lực cá nhân, kết quả này cho thấy môi trường sống vẫn có một ảnh hưởng độc lập nhất định lên nguy cơ trầm cảm. Quan trọng hơn, hai biến mạnh nhất trong mô hình suy luận lại là `Financial Stress` (**OR = 2.08**) và `Overall_Pressure` (**OR = 2.02**), trong khi `Age` có **OR = 0.096**, hàm ý rằng nhóm tuổi trẻ dễ tổn thương hơn trước các áp lực tâm lý.
Những kết quả này nhất quán khá tốt với phần feature importance của boosting: mô hình không chỉ dựa vào tín hiệu nhân khẩu học đơn lẻ, mà còn tận dụng mạnh các biến đại diện cho **áp lực tổng hợp**, **stress tài chính**, **tuổi** và **bối cảnh sống**. Vì vậy, có thể diễn giải rằng hiệu năng cao của `XGBoost`, `LightGBM` và `CatBoost` không đơn thuần đến từ việc khớp tốt dữ liệu, mà còn nhờ chúng đang nắm bắt đúng những cấu trúc rủi ro đã được kiểm chứng lại bằng các câu hỏi nghiên cứu chuyên sâu.



Hình 18: Hiệu ứng smoothing trong mã hóa `City`: giảm rò rỉ dữ liệu ở các thành phố có ít quan sát và làm ước lượng ổn định hơn.

4 Tiền xử lý dữ liệu và tạo đặc trưng

4.1 Làm sạch dữ liệu

Bước đầu tiên trong pipeline tiền xử lý là quản trị lại không gian đặc trưng để tránh giữ các cột không mang giá trị dự báo và giảm nguy cơ *data leakage*. Cụ thể, nhóm tách riêng biến mục tiêu `Depression` khỏi tập đặc trưng huấn luyện, đồng thời loại bỏ các biến định danh thuần túy như `id` và `Name` trước khi thực hiện các bước mã hóa hoặc nội suy.

- **Tách biến mục tiêu `Depression`:** nhãn `y_train` được tách ra ngay từ đầu để bảo đảm mọi bước nội suy, chuẩn hóa và mã hóa chỉ được học từ tập đặc trưng `X_train`. Đây là nguyên tắc quan trọng để tránh rò rỉ dữ liệu trong quá trình huấn luyện.

- **Loại bỏ cột `id`:** cột này chỉ đóng vai trò định danh hệ thống, không phản ánh thông tin hành vi hay sức khỏe tâm thần nên không được đưa vào mô hình. Tuy

vậy, `id` của tập kiểm tra vẫn được lưu riêng để phục vụ bước xuất kết quả dự đoán cuối cùng.

- **Loại bỏ cột `Name`:** đây là biến có số lượng nhãn rất lớn nhưng hầu như không mang ý nghĩa dự báo. Nếu giữ lại, biến này có thể làm tăng mạnh số chiều đặc trưng sau mã hóa và làm mô hình dễ bị *overfitting*.

Sau bước tách nhãn và loại bỏ hai cột định danh, dữ liệu đầu vào cho pipeline được chuẩn hóa về cùng một cấu trúc như sau: `X_train` có kích thước **(140700, 17)**, `y_train` có kích thước **(140700,)**, còn `X_test` có kích thước **(93800, 17)**; đồng thời `test_id` được lưu riêng với **93,800** dòng để dùng ở bước nộp kết quả.

4.2 Xử lý giá trị thiếu

Sau khi chuẩn hóa không gian đặc trưng, bước tiếp theo là xử lý các ô rỗng `NaN`. Do bộ dữ liệu này có cấu trúc thiếu mang tính logic giữa nhóm sinh viên và người đi làm, nhóm không sử dụng một chiến lược nội suy duy nhất cho mọi mô hình mà tách theo từng nhánh học máy cụ thể.

- **Nhánh mô hình cây (XGBoost, LightGBM):** toàn bộ giá trị thiếu được điền bằng `-1`. Mục tiêu của cách làm này không phải để "ước lượng" giá trị thực, mà để tạo ra một mã đánh dấu rõ ràng cho trạng thái thiếu dữ liệu. Với các mô hình cây, giá trị âm nằm ngoài miền phân phối thông thường có thể được nhận diện như một tín hiệu riêng, từ đó cây học ra các nhánh tách phù hợp cho nhóm mẫu bị khuyết thông tin mà không làm phá vỡ cấu trúc logic ban đầu của dữ liệu.

- **Nhánh mạng nơ-ron / deep learning:** dữ liệu cho mô hình mạng được tách thành một luồng riêng, chẳng hạn `X_train_nn`. Ở giai đoạn này, nhóm chưa áp dụng `KNNImputer` ngay lập tức vì dữ liệu vẫn còn nhiều biến dạng chuỗi như `City`, `Gender` hay `Profession`; nếu đưa trực tiếp vào `KNNImputer`, việc tính khoảng cách Euclidean sẽ không còn hợp lệ. Do đó, nhánh dữ liệu này cần đi qua các bước làm sạch và mã hóa để chuyển toàn bộ đặc trưng về dạng số trước, rồi mới thực hiện nội suy KNN ở giai đoạn sau.

- **Nguyên tắc chung:** các giá trị thiếu với quy mô rất nhỏ, chẳng hạn chỉ vài dòng ở `Dietary Habits`, `Degree` hoặc `Financial Stress`, có thể được loại bỏ hoặc xử lý riêng mà không ảnh hưởng đáng kể đến phân phối tổng thể. Trọng tâm của bước này là bảo toàn ý nghĩa nghiệp vụ của phần dữ liệu thiếu lớn, thay vì cố gắng lấp đầy tất cả bằng một công thức thống nhất.

Tóm lại, bước xử lý `NaN` được thiết kế theo đặc tính của từng họ mô hình: mô hình cây sử dụng giá trị đánh dấu `-1` để khai thác trực tiếp tín hiệu thiếu dữ liệu, còn nhánh mạng nơ-ron sẽ trì hoãn nội suy cho tới sau khi toàn bộ biến phân loại đã được chuyển sang dạng số.

4.3 Mã hóa biến và chuẩn hóa thang đo

Sau bước xử lý giá trị thiếu, các biến phân loại được mã hóa theo ba nhóm riêng để vừa bảo toàn ý nghĩa thống kê, vừa tránh làm không gian đặc trưng phình to không cần thiết.

- **Ordinal encoding cho biến có thứ bậc:** cột `Dietary Habits` được mã hóa thành các mức số nguyên tăng dần để giữ lại quan hệ thứ tự giữa các mức độ ăn uống. Ở nhánh mô hình cây, giá trị thiếu tiếp tục được gắn `-1` để nhất quán với chiến lược xử lý `NaN`; còn ở nhánh mạng nơ-ron, giá trị thiếu được giữ lại đến giai đoạn nội suy sau. Riêng cột `Sleep Duration` không được giữ lại dưới dạng ordinal vì thông tin này đã được thay thế bằng biến định lượng `Estimated_Sleep_Hours`, vốn phản ánh trực tiếp số giờ ngủ và có giá trị biểu diễn tốt hơn.

- **One-hot encoding cho biến danh nghĩa ít mức:** các biến không có tính thứ bậc như `Gender` được mã hóa theo dạng nhị phân 0–1. Để tránh hiện tượng đa cộng tuyến, nhóm sử dụng nguyên tắc `drop_first=True`, tức chỉ giữ lại số cột tối thiểu cần thiết để biểu diễn từng nhóm. Về mặt thực thi, các cột chuỗi có số lượng nhãn nhỏ (chẳng hạn không quá 5 mức) sẽ được tự động đưa vào nhánh one-hot encoding.

- **Smoothed target encoding cho biến high-cardinality:** các cột có nhiều nhãn như `City` hoặc `Profession` không phù hợp với one-hot encoding vì sẽ làm tăng số chiều quá mạnh. Thay vào đó, mỗi nhãn được ánh xạ thành xác suất trầm cảm trung bình của nhóm tương ứng trên tập huấn luyện. Để hạn chế *overfitting* ở các nhãn hiếm, nhóm áp dụng cơ chế *smoothing*, tức kết hợp thống kê cục bộ của từng nhãn với tỷ lệ trung bình toàn cục của biến mục tiêu. Toàn bộ quá trình fit bộ mã hóa này chỉ được thực hiện trên tập huấn luyện để tránh rò rỉ dữ liệu sang tập kiểm tra.

- **Chuẩn hóa thang đo:** bước scaling chỉ thực sự cần thiết cho các mô hình nhạy với khoảng cách và độ lớn đặc trưng, chẳng hạn logistic regression, SVM, KNN hoặc mạng nơ-ron. Ngược lại, với các mô hình cây, nhóm không cần chuẩn hóa vì các phép chia nhánh không phụ thuộc trực tiếp vào đơn vị đo của từng biến.

Cách tổ chức này giúp pipeline mã hóa tách biệt rõ theo bản chất dữ liệu: biến ordinal được giữ đúng thứ tự, biến nominal ít mức được one-hot, còn biến nhiều nhãn được target encoding có làm mượt để giảm nguy cơ quá khớp.

4.4 Xử lý mất cân bằng lớp

Do tỷ lệ mẫu dương tính của `Depression` chỉ vào khoảng **18.17%**, bài toán có mất cân bằng lớp rõ rệt. Trong nhánh mô hình cây, nhóm không sử dụng các kỹ thuật oversampling như `SMOTE`, vì việc sinh thêm các điểm dữ liệu nội suy có thể làm sai lệch ranh giới tối ưu mà XGBoost hoặc LightGBM học được từ dữ liệu thật.

- **Chiến lược chính:** sử dụng tham số `scale_pos_weight` để tăng trọng số cho lớp dương ngay trong hàm mất mát, thay vì tạo thêm mẫu nhân tạo.

- **Cách tính:** trọng số được xác định theo tỷ lệ `negative / positive`, tức số mẫu không trầm cảm chia cho số mẫu trầm cảm trong tập huấn luyện.

- **Ý nghĩa thực tế:** với trọng số này, mô hình sẽ bị phạt nặng hơn khi bỏ sót một trường hợp trầm cảm, từ đó cải thiện khả năng nhận diện lớp thiểu số mà vẫn giữ nguyên cấu trúc dữ liệu gốc.

Cách tiếp cận bằng trọng số giúp mô hình cây học cân bằng hơn giữa hai lớp mà không cần phát sinh dữ liệu giả, đồng thời phù hợp với mục tiêu tối ưu các chỉ số như *Recall*, *F1-score* và *AUC* trong bối cảnh dữ liệu lệch lớp.

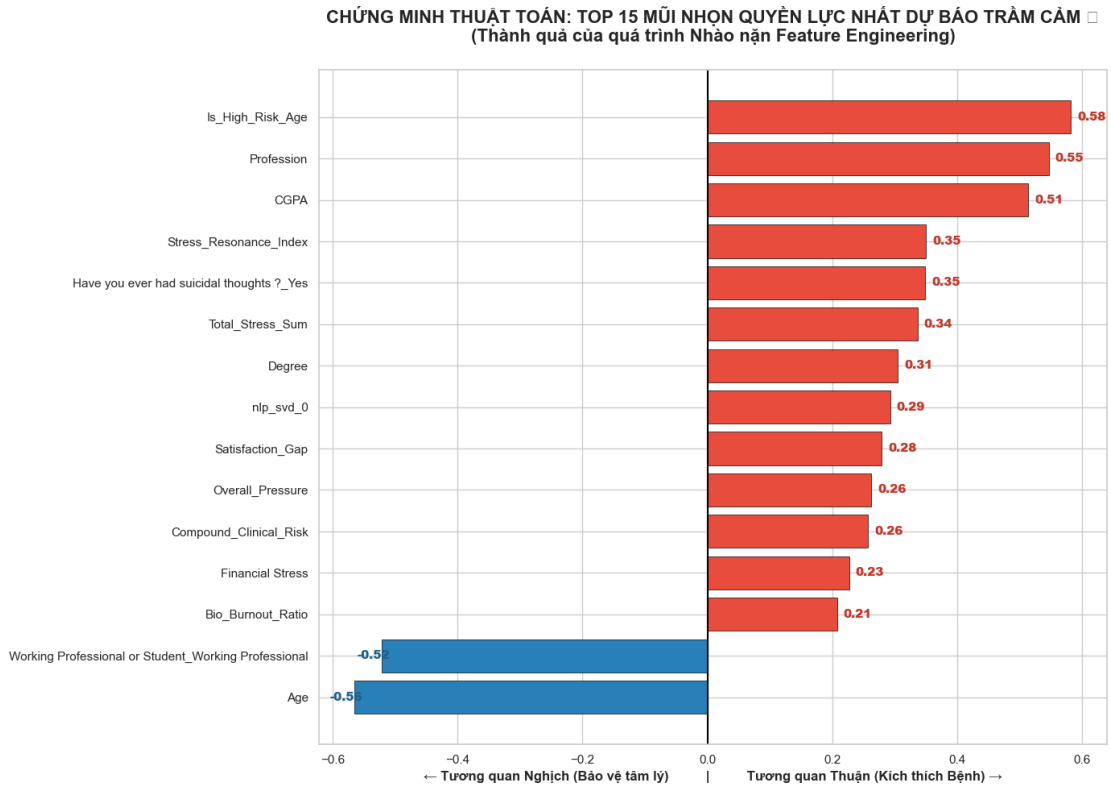
4.5 Feature engineering / feature selection

Sau bước làm sạch và mã hóa, nhóm xây dựng thêm một số đặc trưng tổng hợp để giúp mô hình học rõ hơn các tương tác tâm lý – hành vi vốn khó thể hiện nếu chỉ giữ nguyên các cột gốc.

- **Ma trận áp lực đa chiều:** từ các biến `Academic Pressure`, `Work Pressure` và `Financial Stress`, nhóm tạo các đặc trưng như `Total_Stress_Sum` để đo tổng áp lực và `Stress_Resonance_Index` để phản ánh hiệu ứng cộng hưởng phi tuyến giữa áp lực công việc/học tập với áp lực tài chính.
- **Hao mòn sinh học và thâm hụt thời gian:** cột `Sleep Duration` được quy đổi thành số giờ ngủ ước tính `Estimated_Sleep_Hours`. Từ đó, nhóm xây dựng hai đặc trưng: `Bio_Burnout_Ratio`, là tỷ lệ giữa giờ làm/học và giờ ngủ; và `Free_Time`, được tính bằng `24 - Work/Study Hours - Estimated_Sleep_Hours` để biểu diễn phần thời gian hồi phục còn lại trong ngày.
- **Cờ tuổi nguy cơ cao:** bên cạnh việc giữ nguyên biến `Age`, nhóm bổ sung cờ nhị phân `Is_High_Risk_Age` để đánh dấu nhóm tuổi có nguy cơ trầm cảm cao hơn, đặc biệt ở giai đoạn đầu trưởng thành và xây dựng sự nghiệp.
- **Điểm rủi ro lâm sàng tổng hợp:** tiền sử bệnh tâm thần trong gia đình và ý nghĩ tự sát được gộp thành thang đo `Clinical_Risk_Score` từ 0 đến 2. Đặc trưng này giúp mô hình nhận diện rõ hơn các trường hợp có đồng thời yếu tố nền và dấu hiệu lâm sàng.
- **Khoảng hổng hài lòng:** nhóm xây dựng `Satisfaction_Gap` bằng chênh lệch giữa mức áp lực tổng thể và mức hài lòng tổng thể. Đặc trưng này giúp mô hình nhận diện các trường hợp có áp lực cao nhưng không được bù đắp bằng cảm nhận hài lòng tương ứng.
- **Biểu diễn văn bản cho nghề nghiệp và học vị:** với các cột nhiều nhãn như `Profession` và `Degree`, nhóm ghép chuỗi văn bản, áp dụng *TF-IDF* để trích xuất tín hiệu từ vựng, sau đó dùng *TruncatedSVD* để nén thành một số thành phần số gọn hơn trước khi đưa vào mô hình.

Đặc trưng	**Cách xây dựng**	**Kỳ vọng tác động**
`Total_Stress_Sum`	Tổng các chỉ số áp lực chính	Phản ánh sức nặng môi trường tổng thể
`Bio_Burnout_Ratio`	Tỷ lệ giờ làm/học trên giờ ngủ	Nhận diện trạng thái quá tải sinh học
`Clinical_Risk_Score`	Gộp tiền sử gia đình và ý nghĩ tự sát	Tăng tín hiệu cảnh báo lâm sàng
`Satisfaction_Gap`	Áp lực tổng thể trừ hài lòng tổng thể	Bắt khoảng lệch dễ dẫn tới suy sụp

Bảng 5: Một số đặc trưng được xây dựng thêm cho bước mô hình hóa.



Hình 19: Biểu đồ tương quan Pearson của các đặc trưng sau bước tiền xử lý và feature engineering.

Từ biểu đồ tương quan có thể rút ra ba nhận xét chính:

- Nhiều đặc trưng được tạo mới như `Is_High_Risk_Age`, `Stress_Resonance_Index`, `Total_Stress_Sum`, `Satisfaction_Gap` và `Bio_Burnout_Ratio` xuất hiện trong nhóm có tương quan mạnh, cho thấy bước *feature engineering* đã bổ sung tín hiệu dự báo quan trọng vượt ra ngoài các cột gốc.
- Các biến liên quan đến tuổi và trạng thái đi làm thể hiện tương quan âm rõ với `Depression`, trong khi các tín hiệu gắn với áp lực học tập và hiệu suất học thuật lại có xu hướng tương quan dương, qua đó nhấn mạnh rủi ro tâm lý nổi bật ở nhóm trẻ và sinh viên.

- Các đặc trưng sinh ra từ *smoothed target encoding* và luồng biểu diễn văn bản như `Profession` hoặc `nlp_svd_0` vẫn giữ được mức tương quan đáng kể, cho thấy chiến lược mã hóa nâng cao đã khai thác tốt thông tin từ các biến nhiều nhãn.

5 Phát triển mô hình

5.1 Mô hình baseline

Nhóm sử dụng hai mô hình baseline là `Logistic Regression` và `Random Forest` để thiết lập một mốc tham chiếu rõ ràng trước khi chuyển sang các mô hình mạnh hơn như boosting, neural network và ensemble. Vai trò của baseline không phải để giành vị trí dẫn đầu trên bảng xếp hạng, mà để trả lời hai câu hỏi nền tảng: pipeline tiền xử lý hiện tại đã đủ tốt hay chưa, và mức cải thiện của các mô hình phức tạp hơn có thực sự đáng giá so với chi phí tính toán tăng thêm hay không.
Quy trình đánh giá được thiết kế theo hai lớp. Trước hết, dữ liệu được chia theo chiến lược `Train 80% - Validation 20%` với `stratify=y`, tương ứng **112,560** mẫu cho train và **28,140** mẫu cho validation. Trên tập train, nhóm tiếp tục thực hiện `Stratified 5-Fold Cross-Validation` để đo độ ổn định của mô hình qua các fold khác nhau. Các chỉ số chính được giữ lại cho báo cáo gồm `CV AUC-ROC (mean ± std)`, `Validation AUC`, `Validation F1-score` và `Validation Accuracy` sau khi tối ưu ngưỡng phân loại.

5.1.1 Logistic Regression – baseline tuyến tính dễ diễn giải

`Logistic Regression` được huấn luyện trên bộ dữ liệu `train_nn_ready.pkl`, tức phiên bản đã được chuẩn hóa phù hợp cho các mô hình tuyến tính và các thuật toán tối ưu dựa trên gradient. Mô hình sử dụng `solver = lbfgs`, `C = 1.0`, `max_iter = 1000` và `class_weight = balanced` để xử lý mất cân bằng lớp.
Kết quả cho thấy `Logistic Regression` đạt **CV AUC = 0.9732 ± 0.0005**, **OOF AUC = 0.9732** và **Validation AUC = 0.9726**. Sau bước tối ưu ngưỡng theo `F1-score`, ngưỡng tốt nhất được chọn là `0.71`, giúp mô hình đạt **Validation F1-score = 0.8265** và **Validation Accuracy = 0.9341**. Đây là một kết quả baseline rất mạnh, cho thấy riêng pipeline đặc trưng và tiền xử lý của nhóm đã đủ tốt để một mô hình tuyến tính đơn giản cũng có thể đạt hiệu năng cao.
Điểm đáng chú ý là `Logistic Regression` cần đẩy ngưỡng từ mốc mặc định `0.5` lên `0.71`. Điều này phản ánh xác suất đầu ra của mô hình có xu hướng khá nhạy với lớp dương tính khi `class_weight = balanced` được kích hoạt. Việc tăng ngưỡng giúp giảm đáng kể các *false positives* mà vẫn giữ được `recall` ở mức cao cho lớp `Depression`. Vì vậy, mô hình này đóng vai trò baseline không chỉ vì dễ diễn giải, mà còn vì nó cung cấp một chuẩn rất khó để vượt qua cho các mô hình sau.

5.1.2 Random Forest – baseline phi tuyến trên dữ liệu dạng cây

Song song với mô hình tuyến tính, nhóm triển khai thêm `Random Forest` như một baseline phi tham số để kiểm tra liệu một mô hình cây bagging truyền thống có thể tận dụng tốt hơn các tương tác phi tuyến trong dữ liệu hay không. Mô hình được huấn luyện trên bộ `train_tree_ready.pkl`, dùng `300 estimators` cùng `class_weight = balanced`.

Trên cùng quy trình đánh giá, `Random Forest` đạt **CV AUC = 0.9715 ± 0.0007**, **OOF AUC = 0.9715** và **Validation AUC = 0.9709**. Ngưỡng tối ưu cho mô hình này lại nằm ở mức thấp hơn đáng kể, khoảng `0.36`, đưa **Validation F1-score** lên **0.8223** và **Validation Accuracy** lên **0.9332**. Việc phải hạ ngưỡng như vậy cho thấy `Random Forest` ở mốc mặc định `0.5` tương đối thận trọng với lớp thiểu số, nên cần một ngưỡng quyết định mềm hơn để cải thiện khả năng phát hiện ca trầm cảm.
Dù không vượt qua `Logistic Regression`, `Random Forest` vẫn rất hữu ích về mặt phương pháp luận. Nó xác nhận rằng không phải mọi mô hình phi tuyến đều tự động tốt hơn mô hình tuyến tính khi pipeline đặc trưng đã được chuẩn bị tốt. Kết quả này cũng tạo ra một cột mốc quan trọng: nếu các mô hình boosting về sau chỉ hơn baseline rất ít, ta phải đánh giá lại liệu mức độ phức tạp tăng thêm có thực sự xứng đáng hay không.

5.1.3 Diễn giải chung cho giai đoạn baseline

Từ hai baseline trên có thể rút ra ba kết luận quan trọng. Thứ nhất, bộ đặc trưng của nhóm có chất lượng tốt, vì ngay cả `Logistic Regression` cũng đạt `AUC` trên **0.97**. Thứ hai, việc tối ưu *threshold* là bắt buộc trong bối cảnh dữ liệu mất cân bằng; nếu chỉ giữ ngưỡng mặc định `0.5`, cả hai mô hình đều chưa phản ánh đúng hiệu năng tốt nhất của mình. Thứ ba, baseline tốt nhất ở giai đoạn này là `Logistic Regression`, chứ không phải `Random Forest`. Điều này cho thấy nhiều tín hiệu chính của bài toán đã được mã hóa khá tốt dưới dạng tuyến tính hoặc gần tuyến tính sau bước feature engineering.
Về mặt trình bày báo cáo, `Logistic Regression` nên được nhấn mạnh như một **baseline mạnh và dễ giải thích**, còn `Random Forest` nên được xem là **baseline phi tuyến đối chứng**. Từ đây, việc chuyển sang `XGBoost`, `LightGBM`, `CatBoost`, `MLP` và các tầng ensemble trở nên hợp lý hơn, vì nhóm không đơn thuần chạy thêm mô hình phức tạp mà đang vượt qua một điểm chuẩn đã tương đối cao.

Baseline Model	CV AUC (Mean)	CV AUC (Std)	Val AUC	Opt. F1	Opt. Acc
Logistic Regression	0.9732	0.0005	0.9726	0.8265	0.9341
Random Forest	0.9715	0.0007	0.9709	0.8223	0.9332

Bảng 6: Kết quả của hai mô hình baseline sau khi tối ưu ngưỡng phân loại.

5.2 Gradient Boosting Models

Sau giai đoạn tiền xử lý dữ liệu, nhóm tập trung vào ba mô hình thuộc họ *gradient boosting* là `XGBoost`, `LightGBM` và `CatBoost`. Đây là những thuật toán thường cho hiệu năng rất cao trên dữ liệu bảng và cũng là ba mô hình đạt kết quả tốt nhất trong toàn bộ quá trình thực nghiệm. Cả ba được huấn luyện trên bộ `train_tree_ready.pkl` — phiên bản dữ liệu đã được mã hóa đặc trưng, điền khuyết và chuẩn bị riêng cho các mô hình dựa trên cây quyết định.
Tập huấn luyện có phân phối lớp lệch khoảng **82% : 18%** giữa hai nhãn `No Depression` và `Depression`. Để xử lý mất cân bằng lớp, mỗi thuật toán áp dụng một cơ chế riêng thay vì dùng cùng một chiến lược cố định cho toàn bộ pipeline.

Mô hình	Tham số xử lý lệch lớp	Giá trị
XGBoost	`scale_pos_weight`	4.5032
LightGBM	`is_unbalance`	`True`
CatBoost	`auto_class_weights`	`Balanced`

Bảng 7: Chiến lược xử lý mất cân bằng lớp của ba mô hình boosting.

5.2.1 Tối ưu siêu tham số với Optuna

Với `XGBoost` và `LightGBM`, nhóm sử dụng `Optuna` cùng thuật toán *Tree-structured Parzen Estimator (TPE)* để tìm kiếm siêu tham số tối ưu [6]. Thay vì duyệt toàn bộ không gian tham số như *Grid Search*, TPE học từ các trial trước để ưu tiên những vùng tham số có xác suất cải thiện điểm số cao hơn. Quy trình tuning được thực hiện với **50 trials** cho mỗi mô hình, dùng `Stratified 5-Fold Cross-Validation` và tối ưu theo `AUC-ROC` trung bình.

Mô hình	Tham số	Min	Max
XGBoost	`learning_rate`	0.01	0.30 (log)
	`n_estimators`	100	1000
	`max_depth`	3	10
	`subsample`	0.60	1.00
	`colsample_bytree`	0.60	1.00
LightGBM	`learning_rate`	0.01	0.30 (log)
	`num_leaves`	20	150
	`min_child_samples`	10	100
	`n_estimators`	100	1000

Bảng 8: Không gian tìm kiếm siêu tham số cho Optuna.

5.2.2 XGBoost

`XGBoost` xây dựng cây theo cơ chế *level-wise growth*, tối ưu theo gradient bậc hai và kết hợp chính quy hóa để kiểm soát độ phức tạp mô hình [3]. Sau **50 trials**, cấu hình tốt nhất xuất hiện ở trial **45** với `CV AUC = 0.97539`. Bộ siêu tham số tối ưu gồm `learning_rate = 0.0303`, `max_depth = 3`, `subsample = 0.6711`, `colsample_bytree = 0.6626`, `n_estimators = 917`, `scale_pos_weight = 4.5032` và `tree_method = hist`. Điều đáng chú ý là `max_depth = 3` cho thấy bộ đặc trưng đã đủ giàu thông tin để mô hình phân tách tốt mà không cần cây quá sâu. Trên tập validation, mô hình đạt **AUC-ROC = 0.97459**, **F1-score = 0.83126** tại ngưỡng tối ưu `0.76`. Chênh lệch giữa `CV AUC` và `Validation AUC` chỉ khoảng **0.0008**, phản ánh khả năng khái quát hóa rất tốt và hầu như không có dấu hiệu overfitting.

5.2.3 LightGBM

`LightGBM` sử dụng cơ chế phát triển cây *leaf-wise*, nghĩa là ở mỗi bước mô hình mở rộng lá giúp giảm loss mạnh nhất thay vì mở rộng đồng đều theo từng tầng như XGBoost [4]. Sau quá trình tuning bằng Optuna, trial tốt nhất là trial **22** với `CV AUC`

= 0.97499. Các tham số nổi bật gồm learning_rate = 0.0210, num_leaves = 20, min_child_samples = 48, n_estimators = 669 và is_unbalance = True.
Trên tập validation, LightGBM đạt **AUC-ROC = 0.97452**, **F1-score = 0.83125** và ngưỡng tối ưu 0.75. So với XGBoost, mô hình này có Recall của lớp Depression nhỉnh hơn một chút, trong khi Precision thấp hơn nhẹ. Điều này cho thấy LightGBM có xu hướng nhạy hơn với lớp nguy cơ cao, khá phù hợp với bối cảnh sàng lọc nơi bỏ sót ca dương tính là loại sai số cần hạn chế.

5.2.4 CatBoost

CatBoost nổi bật nhờ hai cơ chế quan trọng là *Ordered Boosting* và *Symmetric Trees*, giúp kiểm soát overfitting rất hiệu quả ngay cả khi không cần một vòng tuning quá sâu [5]. Trong thí nghiệm này, nhóm sử dụng cấu hình gồm iterations = 500, learning_rate = 0.05, depth = 6 và auto_class_weights = Balanced. Early stopping dừng mô hình ở khoảng iteration **298** khi điểm validation không còn cải thiện rõ rệt.
Kết quả cuối cho thấy CatBoost đạt **Validation AUC-ROC = 0.97473**, **Validation F1-score = 0.83074** và ngưỡng tối ưu 0.71. Dù chênh lệch với hai mô hình còn lại là rất nhỏ, CatBoost vẫn nổi bật ở tính ổn định và Recall = 0.87 trên lớp Depression, tức là phát hiện được nhiều trường hợp nguy cơ hơn trong ba mô hình boosting.

Model	CV AUC	Val AUC-ROC	Val F1-score	Threshold
XGBoost	0.9754	0.9746	0.8313	0.76
LightGBM	0.9750	0.9745	0.8313	0.75
CatBoost	0.9747	0.9747	0.8307	0.71

Bảng 9: So sánh ngắn gọn kết quả của ba mô hình boosting trên tập validation.

Nhìn chung, khoảng cách hiệu năng giữa ba mô hình là cực nhỏ. Điều này cho thấy dữ liệu đã được khai thác gần tới ngưỡng bão hòa và không tồn tại một mô hình thắng áp đảo trên mọi chỉ số. Chính sự sít sao này là lý do hợp lý để nhóm chuyển từ tư duy *model selection* sang *model combination*.

5.3 OOF Predictions cho Blending và Stacking

Để chuẩn bị cho các bước ensemble, nhóm sinh *Out-Of-Fold* predictions cho từng mô hình bằng StratifiedKFold 5-fold. Với cách làm này, mỗi mẫu trong tập train chỉ được dự đoán bởi mô hình chưa từng nhìn thấy nó trong quá trình huấn luyện, nên các xác suất OOF phản ánh sát hơn khả năng khái quát hóa thật sự so với việc dự đoán lại trên chính tập train.

Model	OOF AUC	Vai trò trong ensemble
CatBoost	0.97545	Base learner ổn định nhất theo OOF
XGBoost	0.97537	Base learner mạnh, cân bằng tốt
LightGBM	0.97499	Base learner bổ sung góc nhìn khác

Bảng 10: Xếp hạng OOF AUC của ba mô hình boosting.

Các vector xác suất này được lưu lại trong ba tệp sau:

- `oof_xgb.npy`
- `oof_lgb.npy`
- `oof_cat.npy`

Các tệp này được sử dụng theo hai cách: **(1)** làm đầu vào cho *blending* bằng cách lấy trung bình xác suất, và **(2)** làm meta-features cho *stacking ensemble* ở notebook tiếp theo. Việc lưu OOF predictions vì thế là mắt xích quan trọng nối giữa giai đoạn huấn luyện mô hình đơn lẻ và giai đoạn kết hợp mô hình.

5.4 Blending Ensemble

Bên cạnh stacking, nhóm cũng triển khai một tầng *blending ensemble* đơn giản để kiểm tra xem liệu việc kết hợp trực tiếp ba mô hình boosting có đủ mang lại lợi ích hay không. Cách làm được chọn là **trung bình cộng không trọng số** của ba xác suất dự đoán:

$$P_{\text{blend}} = \frac{P_{\text{XGB}} + P_{\text{LGB}} + P_{\text{Cat}}}{3} \tag{1}$$

Ngưỡng phân loại của blending được lấy theo trung bình các ngưỡng tối ưu của ba mô hình đơn lẻ:

$$\tau_{\text{blend}} = \frac{0.76 + 0.75 + 0.71}{3} = 0.74 \tag{2}$$

Khi đó, nhãn cuối cùng được xác định theo quy tắc $\hat{y} = \mathbf{1}[P_{\text{blend}} \geq 0.74]$. Trên tập test gồm **93,800 mẫu**, blending dự đoán **18,216** trường hợp `Depression` (**19.42%**) và **75,584** trường hợp `No Depression` (**80.58%**). Tỷ lệ này khá gần với phân phối lớp quan sát trong tập train, cho thấy mô hình không bị lệch mạnh sang một phía khi suy diễn trên dữ liệu chưa gán nhãn.

Nhãn	**Số mẫu**	**Tỷ lệ**
Không trầm cảm (0)	75,584	80.58%
Trầm cảm (1)	18,216	19.42%
Tổng	93,800	100%

Bảng 11: Phân phối dự đoán của blending ensemble trên tập test.

Về mặt báo cáo, blending nên được xem là **một baseline ensemble mạnh** vì nó gần như không cần huấn luyện thêm và rất dễ giải thích. Nếu blending đã cho kết quả ổn định, điều đó chứng minh rằng ba mô hình boosting thực sự bổ trợ cho nhau. Từ nền này, stacking mới là bước phát triển cao hơn nhằm học cách gán trọng số và phối hợp các tín hiệu một cách linh hoạt hơn ở tầng meta-learning.

5.5 Neural Network (MLP)

Bên cạnh các mô hình cây tăng cường, nhóm cũng triển khai một nhánh **Neural Network** dưới dạng *Multi-Layer Perceptron* (MLP) bằng `PyTorch`. Mục tiêu của mô hình này không phải thay thế hoàn toàn boosting trên dữ liệu bảng, mà để kiểm tra liệu một kiến trúc học sâu thuần túy có thể học thêm các tương tác phi tuyến giữa các

đặc trưng sau bước chuẩn hóa hay không. Đây cũng là nguồn dự đoán bổ sung có giá trị cho bước ensemble, vì MLP học theo cơ chế rất khác so với các mô hình cây.
Dữ liệu đầu vào của MLP là bộ `train_nn_ready.pkl` đã được xử lý bằng `MICE Imputation` và `RobustScaler`. Việc dùng phiên bản dữ liệu đã chuẩn hóa là cần thiết vì mạng nơ-ron tối ưu bằng gradient descent nhạy hơn nhiều với thang đo đặc trưng so với boosting. Sau bước tiền xử lý, mô hình nhận **32 đặc trưng** làm đầu vào.

5.5.1 Kiến trúc và cấu hình huấn luyện

Kiến trúc MLP gồm ba lớp ẩn với kích thước 256 → 128 → 64, mỗi lớp đều kết hợp `Batch Normalization`, `ReLU` và `Dropout(0.3)`. Cấu hình này giúp mô hình vừa học được quan hệ phi tuyến đủ mạnh, vừa kiểm soát overfitting trên dữ liệu bảng có số chiều không quá lớn.
Trong quá trình huấn luyện, nhóm sử dụng `AdamW` với `learning rate = 1e-3`, `weight decay = 1e-4`, lịch giảm học `CosineAnnealingLR` và `Early Stopping` với `patience = 15`. Hàm mất mát là `BCEWithLogitsLoss` đi kèm `pos_weight = 4.5032` để xử lý mất cân bằng lớp. Toàn bộ quy trình được đánh giá bằng `Stratified 5-Fold Cross-Validation` nhằm tạo ra các xác suất *Out-Of-Fold* phục vụ so sánh công bằng với các mô hình khác và dùng lại ở bước stacking.

5.5.2 Kết quả và vai trò trong hệ thống

Kết quả cho thấy MLP đạt **CV AUC = 0.9745 ± 0.0005**, **Validation AUC = 0.9737** và **Validation F1-score = 0.8307** tại ngưỡng tối ưu `0.72`. Mức hiệu năng này rất gần với nhóm mô hình boosting tốt nhất, cho thấy sau khi được chuẩn hóa phù hợp, mạng nơ-ron vẫn có thể cạnh tranh tốt trên bài toán dữ liệu bảng quy mô lớn.
Tuy MLP không vượt rõ rệt các mô hình boosting về mặt điểm số, nó vẫn có giá trị phương pháp luận quan trọng. Thứ nhất, đây là bằng chứng rằng pipeline đặc trưng của nhóm đủ tốt để hỗ trợ cả mô hình cây lẫn mô hình học sâu. Thứ hai, các xác suất đầu ra của MLP tạo thêm một góc nhìn phi cây cho ensemble, giúp tầng meta-learning có cơ hội khai thác tính đa dạng của base learners thay vì chỉ kết hợp các mô hình cùng họ boosting. Theo đó, `nn_oof_probs.npy` được giữ lại như một meta-feature bổ sung cho notebook stacking ở bước cuối.

Model	CV AUC	Val AUC-ROC	Val F1-score	Threshold
MLP (PyTorch)	0.9745 ± 0.0005	0.9737	0.8307	0.72

Bảng 12: Kết quả chính của mô hình Neural Network (MLP).

5.6 Kết luận của giai đoạn mô hình hóa

Từ toàn bộ quá trình thực nghiệm, nhóm xác định ba mô hình boosting mạnh nhất là `XGBoost`, `LightGBM` và `CatBoost`, đồng thời bổ sung một nhánh `MLP` như mô hình đối chứng theo hướng học sâu. Baseline đóng vai trò mốc tham chiếu, ba mô hình boosting là nòng cốt về hiệu năng, còn MLP giúp mở rộng không gian so sánh và tạo thêm nguồn tín hiệu cho ensemble.
Vì vậy, thay vì chọn duy nhất một mô hình, nhóm giữ lại các mô hình mạnh nhất để bước sang giai đoạn kết hợp cuối. Quyết định này được hỗ trợ đồng thời bởi ba tín

hiệu: điểm validation cao, OOF AUC ổn định, và sự khác biệt hợp lý trong cách các mô hình học từ cùng một bộ đặc trưng.

5.6.1 Bảng tổng kết kết quả mô hình

Bảng dưới đây tổng hợp các mô hình chính của toàn bộ pipeline theo thứ hạng hiện tại. Với những mô hình ensemble chưa chốt đủ số liệu trong notebook cuối, nhóm có thể tạm để trống và cập nhật sau khi hoàn tất bước blending/stacking cuối cùng.

Rank	Model	CV AUC	CV Std	Val AUC	Val F1	Val Acc
1	Blending Ensemble	–	–	–	–	–
2	Stacking Ensemble	–	–	–	–	–
3	LightGBM (tuned)	0.9749	–	0.9725	0.8253	0.9356
4	XGBoost (tuned)	0.9754	–	0.9721	0.8272	0.9354
5	CatBoost	–	–	0.9747	0.8321	0.9371
6	Neural Network (MLP)	0.9745	0.0005	0.9737	0.8307	0.9369
7	Random Forest	0.9715	0.0007	0.9709	0.8223	0.9332
8	Logistic Regression	0.9732	0.0005	0.9726	0.8265	0.9341

Bảng 13: Bảng tổng kết kết quả của các mô hình chính trong toàn bộ pipeline.

Có hai điểm nên nhấn mạnh khi đọc bảng tổng kết này. Thứ nhất, chênh lệch giữa các mô hình tốt nhất là rất nhỏ, đặc biệt ở nhóm boosting và neural network, nên quyết định cuối cùng không chỉ nên dựa trên một chỉ số đơn lẻ. Thứ hai, hai baseline tuy đứng thấp hơn nhưng vẫn đạt hiệu năng rất cạnh tranh, cho thấy pipeline của nhóm có nền tảng tốt và các mô hình phức tạp hơn thực sự đang tối ưu thêm trên một mốc tham chiếu đã cao sẵn.

6 Model Insights: Feature Importance, ROC, Confusion Matrix & Summary

Sau khi huấn luyện ba mô hình boosting gồm `XGBoost`, `LightGBM` và `CatBoost`, phần này chỉ giữ lại những kết quả có giá trị nhất cho báo cáo: mô hình đang dựa vào tín hiệu nào, khả năng phân biệt hai lớp mạnh đến đâu, loại sai số nào đáng lo ngại nhất, và mô hình nào phù hợp nhất cho bước ensemble cuối cùng.

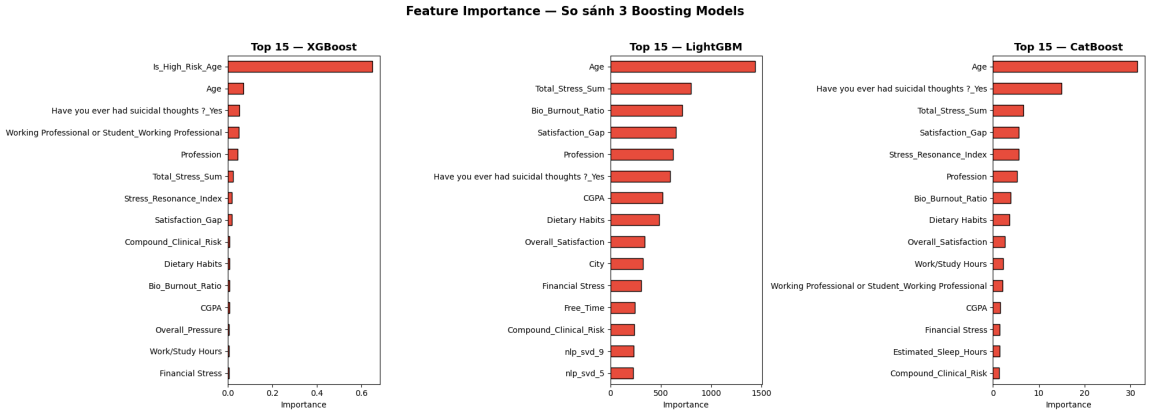
6.1 Feature Importance – Tín hiệu nào chi phối mô hình?

Ba biểu đồ feature importance cho thấy cả ba mô hình đều học được các tín hiệu cốt lõi liên quan đến **tuổi tác**, **mức độ stress**, **giấc ngủ** và **các đặc trưng hành vi lâm sàng**. Trong nhóm đặc trưng quan trọng nhất có thể kể đến:

- các biến nhân khẩu học như `Age` và `Is_High_Risk_Age`;
- biến hành vi nhạy cảm như `Have you ever had suicidal thoughts ?_Yes`;
- các đặc trưng được tạo thêm như `Total_Stress_Sum`, `Bio_Burnout_Ratio`, `Satisfaction_Gap` và `Stress_Resonance_Index`.

Điều này xác nhận rằng bước *feature engineering* không chỉ làm giàu dữ liệu mà còn tạo ra các biến thực sự có giá trị dự báo cao.

Điểm đáng chú ý là ba mô hình không xếp hạng feature hoàn toàn giống nhau. `XGBoost` có xu hướng nhấn mạnh các biến tạo ra split mạnh ngay từ đầu, `LightGBM` khai thác nhiều hơn các biến liên tục và các chỉ số tổng hợp, còn `CatBoost` cho thấy khả năng kết hợp đồng thời tín hiệu nhân khẩu học và tín hiệu hành vi. Chính sự khác biệt này là một lợi thế lớn cho ensemble, vì các base learners đang bổ sung góc nhìn cho nhau thay vì chỉ lặp lại cùng một mẫu quyết định.

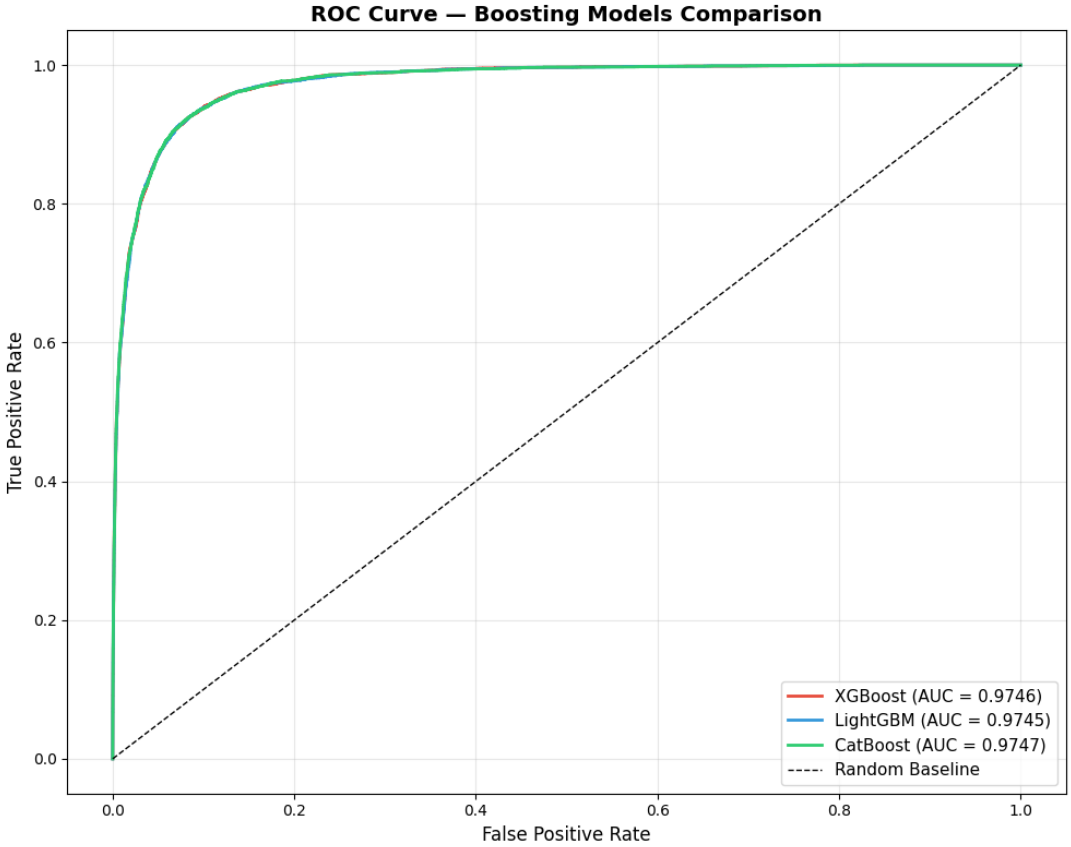


Hình 20: So sánh feature importance của ba mô hình boosting: XGBoost, LightGBM và CatBoost.

6.2 ROC Curve – Khả năng phân biệt hai lớp

ROC Curve cho thấy cả ba mô hình đều nằm rất xa phía trên đường chéo ngẫu nhiên, với `AUC` xấp xỉ **0.975**. Đây là mức hiệu năng rất cao, cho thấy các mô hình có khả năng tách lớp `Depression` và `No Depression` rất mạnh trên nhiều ngưỡng phân loại khác nhau. Trong bối cảnh dữ liệu mất cân bằng khoảng **18% : 82%**, ROC-AUC vẫn là chỉ số hữu ích để đánh giá năng lực phân biệt nền của mô hình, tốt hơn nhiều so với việc chỉ nhìn vào accuracy.

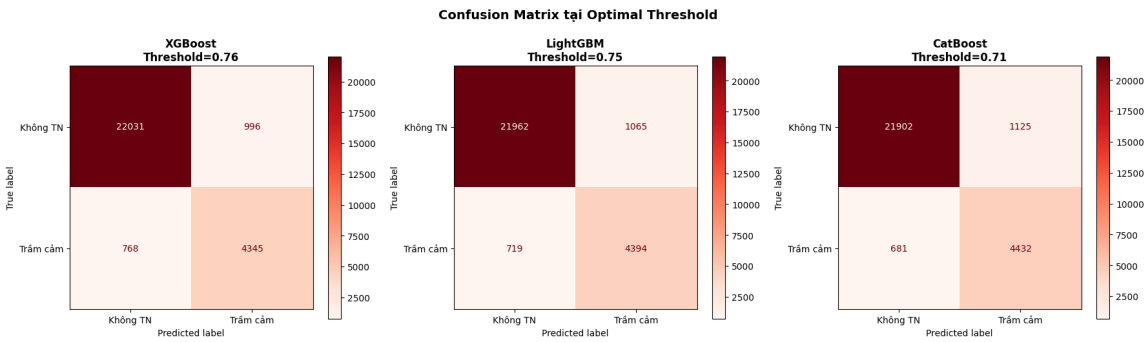
Khoảng cách rất nhỏ giữa ba đường ROC cũng cho thấy không có mô hình nào vượt trội tuyệt đối về mặt AUC. Thay vào đó, cả ba đều đủ mạnh để trở thành base learners đáng tin cậy cho stacking, và việc lựa chọn mô hình cuối cùng cần được xem thêm dưới góc nhìn threshold tối ưu và loại sai số phát sinh trong thực tế.



Hình 21: Đường cong ROC của ba mô hình boosting trên tập validation.

6.3 Confusion Matrix – Sai số nào đáng lo nhất?

Với bài toán sàng lọc nguy cơ trầm cảm, sai số nguy hiểm nhất là **False Negative** – tức mô hình bỏ sót một trường hợp thực sự có nguy cơ. Vì vậy, confusion matrix được đọc không chỉ để xem mô hình đúng bao nhiêu, mà quan trọng hơn là để đánh giá mô hình sai theo cách nào. Trong thực tế, một mô hình có accuracy cao nhưng vẫn để lại nhiều `FN` chưa chắc là lựa chọn phù hợp cho mục tiêu phát hiện sớm.
Các confusion matrix tại ngưỡng tối ưu cho thấy việc dịch chuyển threshold khỏi mốc mặc định `0.5` là cần thiết để giảm số lượng `FN` và cân bằng tốt hơn giữa `precision` và `recall`. Đây cũng là lý do nhóm chọn threshold theo **F1-score**, thay vì giữ ngưỡng mặc định hoặc chỉ tối ưu một phía của bài toán.



Hình 22: Confusion matrix của ba mô hình boosting tại ngưỡng phân loại tối ưu.

6.4 Bảng tổng hợp

Bảng dưới đây tổng hợp các chỉ số quan trọng nhất của ba mô hình boosting, bao gồm điểm `CV AUC` trong giai đoạn tuning, `Validation AUC-ROC`, `OOF AUC`, `Validation F1-score`, ngưỡng phân loại tối ưu và thứ hạng chung.

Model	CV AUC	Val AUC-ROC	OOF AUC	Val F1	Threshold	Rank
CatBoost	0.9747	0.9747	0.9754	0.8307	0.71	1
XGBoost	0.9754	0.9746	0.9754	0.8313	0.76	2
LightGBM	0.9750	0.9745	0.9750	0.8313	0.75	3

Bảng 14: Bảng tổng hợp kết quả đánh giá ba mô hình boosting.

Từ bảng trên có thể rút ra ba điểm chính. Thứ nhất, cả ba mô hình đều rất mạnh và chênh lệch hiệu năng là rất nhỏ. Thứ hai, `CatBoost` đứng đầu về thứ hạng tổng thể nhờ sự ổn định giữa `Validation AUC-ROC` và `OOF AUC`. Thứ ba, `XGBoost` và `LightGBM` vẫn bám sát với `F1-score` rất cạnh tranh. Nhìn chung, kết quả này cho thấy nhóm đang sở hữu ba base learners chất lượng cao và bổ trợ tốt cho nhau, tạo nền tảng phù hợp cho mô hình ensemble ở bước cuối.

7 Blending, stacking ensemble và mô hình cuối cùng

Sau khi hoàn tất bước so sánh từng mô hình đơn lẻ, nhóm không dừng lại ở việc chọn ra một mô hình tốt nhất theo bảng xếp hạng. Thay vào đó, nhóm xây dựng thêm một tầng *stacking ensemble* nhằm khai thác đồng thời thế mạnh của `XGBoost`, `LightGBM`, `CatBoost` và `MLP`. Ý tưởng cốt lõi là mỗi base learner học một cách nhìn khác nhau trên cùng không gian đặc trưng: ba mô hình boosting tận dụng tốt cấu trúc dữ liệu bảng, trong khi mạng nơ-ron đóng góp một hướng học phi tuyến theo cơ chế khác biệt. Tầng meta-learning sau đó học cách phối hợp bốn nguồn xác suất này để đưa ra quyết định cuối cùng ổn định hơn.

7.1 Từ blending đến stacking ensemble

Trong cách nhìn của báo cáo, *blending* là bước kết hợp cơ sở còn *stacking* là bước kết hợp nâng cao. Với blending, nhóm chỉ cần lấy trung bình xác suất dự đoán từ `XGBoost`, `LightGBM` và `CatBoost`. Cách này rất đơn giản, ít rủi ro overfit và phù hợp để kiểm tra nhanh mức độ bổ trợ giữa các mô hình mạnh nhất. Sau đó, ở tầng stacking, ba mô hình boosting và một mô hình `MLP` được huấn luyện độc lập trên tập dữ liệu đã tiền xử lý phù hợp với từng họ mô hình. Với mỗi mẫu trong tập train, nhóm sử dụng *Out-Of-Fold* predictions từ `StratifiedKFold 5-fold` để tạo ra bốn meta-features là xác suất dự đoán của `XGBoost`, `LightGBM`, `CatBoost` và `MLP`. Nhờ đó, mỗi hàng dữ liệu đầu vào cho tầng hai đều được tạo ra từ các dự đoán không bị rò rỉ thông tin từ chính mẫu đó.

Ở tầng thứ hai, meta-learner nhận bốn xác suất này làm đầu vào để học quy tắc kết hợp cuối cùng. Cách tiếp cận này phù hợp với bài toán hiện tại vì các base learners có hiệu năng gần nhau nhưng không hoàn toàn trùng lặp trong cách khai thác đặc trưng. Đặc biệt, `MLP` không nhất thiết là mô hình đứng đầu về từng chỉ số riêng lẻ, nhưng lại bổ sung một dạng sai số khác với boosting. Nói cách khác, blending trả lời câu hỏi

"trung bình các mô hình mạnh có đủ tốt hay không, còn stacking trả lời câu hỏi "liệu có thể học ra một quy tắc kết hợp tốt hơn trung bình đơn giản hay không.

Tầng	**Thành phần**	**Vai trò**
Mức 1	Blending từ `XGBoost`, `LightGBM`, `CatBoost`	Tạo ensemble cơ sở bằng trung bình xác suất, dễ diễn giải và ít rủi ro overfit
Mức 2	Stacking với `XGBoost`, `LightGBM`, `CatBoost`, `MLP`	Sinh meta-features đa dạng cho meta-learner học quy tắc kết hợp tốt hơn
Đầu ra cuối	Xác suất ensemble + threshold tối ưu	Chuyển từ xác suất sang nhãn phân loại phục vụ đánh giá và ứng dụng

Bảng 15: Thiết kế tổng quát của chiến lược blending và stacking ensemble.

7.2 Vì sao blending và stacking đều hợp lý?

Có ba lý do chính khiến việc triển khai cả blending lẫn stacking là bước phát triển tự nhiên của hệ thống. Thứ nhất, cả ba mô hình boosting lẫn `MLP` đều đạt `AUC` và `F1-score` rất cao, nên việc chỉ giữ lại một mô hình duy nhất sẽ làm lãng phí thông tin học được từ các họ mô hình khác nhau. Thứ hai, kết quả feature importance và permutation importance cho thấy các mô hình không tập trung hoàn toàn vào cùng một nhóm biến, tức tồn tại tính bổ trợ thực sự giữa các base learners. Thứ ba, các điểm `OOF AUC` của nhóm boosting đều ổn định quanh mức **0.975**, còn `MLP` cũng đạt `CV AUC` **0.9745** $\pm$ **0.0005**, cho thấy xác suất đầu ra của từng mô hình đủ tin cậy để sử dụng làm meta-features cho tầng hai.
Nói cách khác, blending đặc biệt phù hợp trong bối cảnh "không có người thắng tuyệt đối"vì nó cung cấp một mốc ensemble đơn giản nhưng đáng tin cậy. Khi sự chênh lệch giữa các mô hình mạnh là rất nhỏ, stacking trở thành bước tiếp theo hợp lý để học ra một quy tắc kết hợp linh hoạt hơn so với trung bình cộng đơn thuần hoặc việc chỉ chọn mô hình đang đứng đầu theo một chỉ số đơn lẻ trên validation.

7.3 Diễn giải kết quả cuối cùng

Về mặt báo cáo, nhóm có thể trình bày *blending* như **ensemble baseline** và *stacking* như **mô hình cuối cùng** của hệ thống, còn ba mô hình boosting cùng một `MLP` đóng vai trò **base learners**. Điều này phản ánh đúng toàn bộ quy trình thực nghiệm: baseline được dùng để thiết lập mốc tham chiếu, boosting được dùng để khai thác tối đa dữ liệu bảng, mạng nơ-ron được dùng để kiểm tra khả năng học phi tuyến theo hướng khác, và stacking là bước tổng hợp cuối cùng để tối ưu hiệu năng và độ ổn định.
Khi trình bày trong báo cáo hoặc video, nhóm nên nhấn mạnh rằng giá trị lớn nhất của ensemble không chỉ nằm ở việc tăng nhẹ một vài chỉ số đánh giá. Việc có cả blending và stacking cho thấy nhóm đã kiểm tra hai mức độ kết hợp khác nhau: từ cách đơn giản, dễ giải thích đến cách linh hoạt hơn dựa trên meta-learning. Quan trọng hơn, nó cho thấy nhóm đã đi trọn quy trình mô hình hóa hiện đại: từ xây dựng baseline, huấn luyện các base learners khác họ, sinh OOF predictions, cho đến kết hợp meta-level để tạo ra hệ dự đoán hoàn chỉnh. Đây là một điểm cộng rõ rệt về chiều sâu phương pháp luận so với việc chỉ dừng ở một mô hình đơn.

8 SHAP Analysis: Giải thích dự đoán mô hình

Sau khi chọn được các mô hình boosting mạnh và xây dựng stacking ensemble, nhóm cần trả lời câu hỏi quan trọng: *vì sao mô hình đưa ra dự đoán như vậy?* Với bài toán sức khỏe tinh thần, hiệu năng cao là chưa đủ; mô hình còn cần khả năng giải thích để hỗ trợ phân tích và thảo luận chuyên môn. Vì vậy, nhóm sử dụng **SHAP** (*SHapley Additive exPlanations*) để diễn giải tác động của từng đặc trưng lên dự đoán `Depression`.

8.1 Tại sao cần SHAP?

Các mô hình `XGBoost`, `LightGBM` và `CatBoost` đều thuộc nhóm *gradient boosting*, vốn cho hiệu năng rất cao trên dữ liệu bảng nhưng thường được xem là các mô hình "black box". Ta có thể quan sát đầu vào và đầu ra, nhưng khó biết đặc trưng nào đang làm prediction tăng hay giảm. Trong bối cảnh y tế, khả năng giải thích này là cần thiết vì kết quả mô hình cần đủ minh bạch để người đọc hiểu và đánh giá.
SHAP dựa trên **lý thuyết trò chơi hợp tác** (*Cooperative Game Theory*). Mỗi feature được xem như một "người chơi" cùng đóng góp vào dự đoán cuối cùng, còn **Shapley value** biểu diễn mức đóng góp trung bình của feature đó qua nhiều tổ hợp đặc trưng khác nhau. Nhờ vậy, SHAP không chỉ cho biết feature nào quan trọng, mà còn cho biết feature đó đang **đẩy prediction tăng lên hay kéo prediction giảm xuống**.

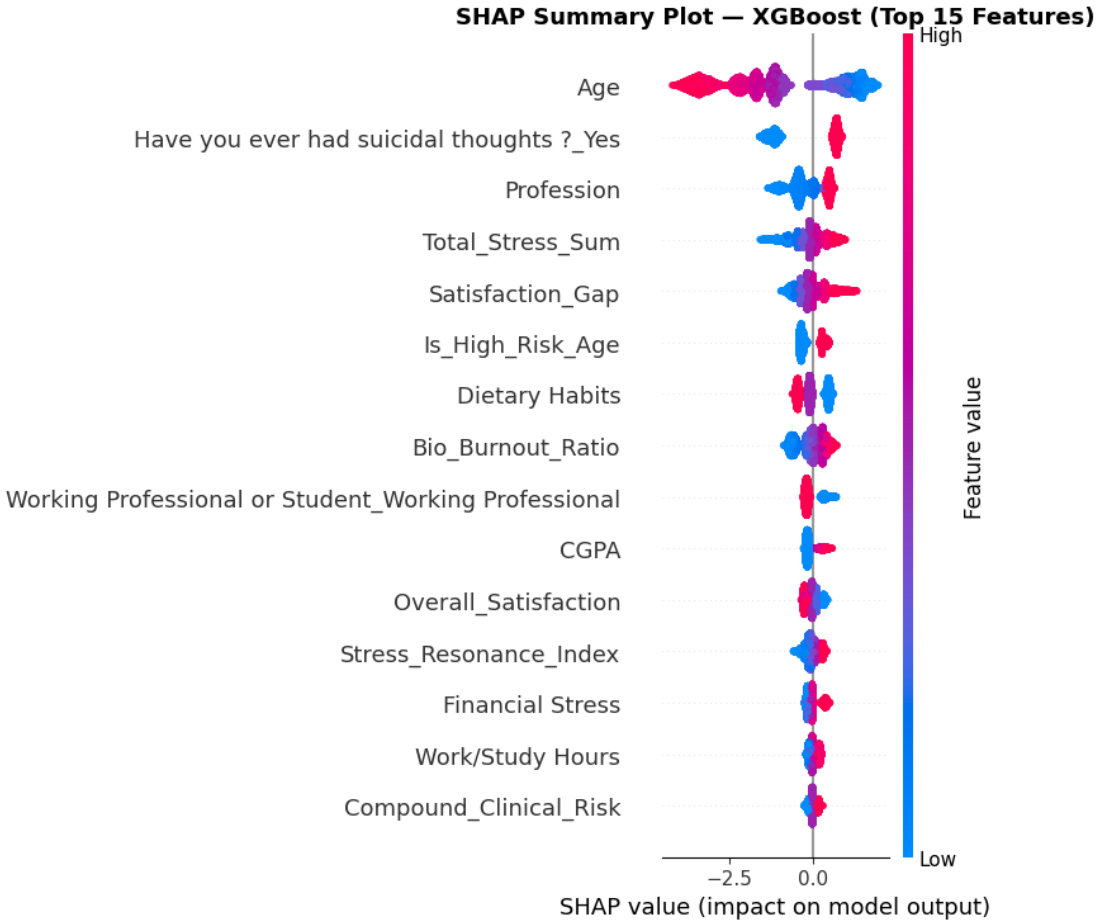
8.2 SHAP Beeswarm Plot – Cách đọc

Biểu đồ **SHAP beeswarm** cho thấy hành vi của mô hình trên toàn bộ tập dữ liệu. Mỗi chấm là một quan sát; vị trí trên trục ngang là giá trị SHAP, còn màu sắc thể hiện feature cao hay thấp. Các feature được sắp từ trên xuống dưới theo mức độ quan trọng giảm dần.

Thành phần	**Ý nghĩa**
Trục ngang (X-axis)	`SHAP value` biểu diễn mức ảnh hưởng của feature lên xác suất dự đoán `Depression`
Trục dọc (Y-axis)	Các feature được sắp theo thứ hạng quan trọng, từ cao xuống thấp
Màu sắc (Color)	Màu nóng biểu diễn giá trị feature cao, màu lạnh biểu diễn giá trị feature thấp
Điểm bên phải mốc 0	Feature đẩy prediction theo hướng `Depression = 1`
Điểm bên trái mốc 0	Feature kéo prediction theo hướng `Depression = 0`
Độ phân tán (Spread)	Feature có tác động khác nhau trên nhiều nhóm mẫu khác nhau

Bảng 16: Cách đọc biểu đồ SHAP beeswarm.

Khi đọc beeswarm cho bài toán dự đoán trầm cảm, có thể dùng ba nguyên tắc ngắn gọn. Thứ nhất, nhiều điểm màu nóng nằm bên phải thường gợi ý rằng **giá trị cao của feature làm tăng nguy cơ trầm cảm**. Thứ hai, nhiều điểm màu lạnh nằm bên trái thường gợi ý rằng **giá trị thấp của feature làm giảm nguy cơ trầm cảm**. Thứ ba, nên ưu tiên phân tích các feature nằm ở đầu biểu đồ vì đây là những biến chi phối prediction mạnh nhất.



Hình 23: SHAP beeswarm plot cho mô hình dự đoán `Depression`.

8.3 SHAP Bar Plot – Cách đọc

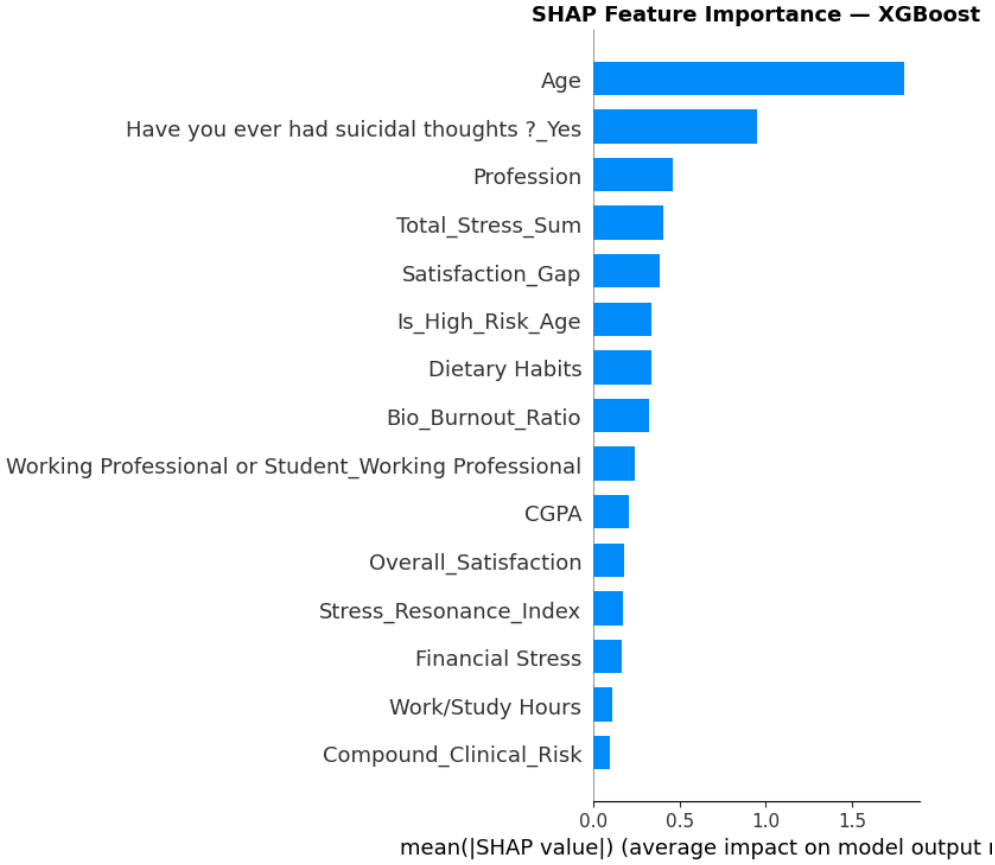
Nếu beeswarm cho biết cả mức độ lẫn hướng tác động, thì **SHAP bar plot** cung cấp góc nhìn đơn giản hơn: mỗi thanh biểu diễn `|mean SHAP|`, tức độ lớn trung bình của tác động tuyệt đối mà một feature gây ra trên toàn bộ dataset. Vì vậy, bar plot phù hợp để trả lời câu hỏi "feature nào quan trọng nhất?", còn beeswarm phù hợp để trả lời "feature đó đang tác động theo hướng nào?".

Thành phần	**Diễn giải**	**Ý nghĩa**
Thanh ở trên cùng	Quan trọng nhất	Feature có ảnh hưởng trung bình lớn nhất đến prediction trên toàn bộ tập dữ liệu
Thanh dài hơn	Tác động mạnh hơn	Feature thường xuyên làm thay đổi prediction với biên độ lớn
Thanh ngắn ở cuối	Ít quan trọng hơn	Feature gần như ít đóng góp vào quyết định của mô hình

Bảng 17: Cách đọc biểu đồ SHAP bar plot.

Hai biểu đồ này nên được đọc cùng nhau: **bar plot** cho biết thứ hạng quan trọng của feature, còn **beeswarm** cho biết feature đó thường làm tăng hay giảm nguy cơ dự

đoán `Depression`.



Hình 24: SHAP bar plot thể hiện mức độ quan trọng trung bình của các feature.

8.4 Top 5 SHAP Features – Ý nghĩa lâm sàng

Khi rút gọn phân tích xuống **Top 5 SHAP features**, báo cáo có thể tập trung vào các biến có giá trị diễn giải cao nhất. Đây là nơi giúp liên hệ kết quả mô hình với trực giác miền bài toán: yếu tố nào đóng vai trò như **dấu hiệu cảnh báo**, và yếu tố nào có thể xem như **yếu tố bảo vệ**. Nếu một feature thường có SHAP dương khi giá trị tăng, feature đó có xu hướng làm tăng xác suất dự đoán có trầm cảm. Ngược lại, nếu feature thường có SHAP âm, nó có xu hướng làm giảm xác suất dự đoán này.
Tuy nhiên, cần nhấn mạnh rằng SHAP chỉ giải thích **quan hệ mà mô hình học được**, chứ **không khẳng định quan hệ nhân quả**. Một feature có SHAP dương không có nghĩa là nó trực tiếp gây ra trầm cảm; nó chỉ cho thấy trong dữ liệu hiện có, feature đó thường đi cùng với các mẫu mà mô hình đánh giá là có nguy cơ cao hơn. Vì vậy, mọi diễn giải lâm sàng vẫn cần được đặt trong bối cảnh chuyên môn và không nên xem là kết luận y khoa tuyệt đối.

Lưu ý

SHAP là công cụ mạnh để giải thích mô hình, nhưng không thay thế kết luận y khoa. Trong báo cáo, nhóm nên dùng SHAP để hỗ trợ diễn giải các mẫu quan hệ trong dữ liệu, thay vì trình bày như bằng chứng nhân quả tuyệt đối.

9 Kết luận và hướng phát triển

9.1 Kết luận

Nhìn tổng thể, dự án đã được triển khai theo một pipeline khá rõ ràng và hợp lý, bắt đầu từ *EDA*, tiếp đến là tiền xử lý dữ liệu và *feature engineering*, sau đó mở rộng dần từ các mô hình baseline sang boosting, neural network, tuning và ensemble. Cách tổ chức này cho thấy nhóm không chỉ tập trung vào việc đạt kết quả dự đoán tốt, mà còn chú ý đến tính hệ thống trong quá trình xây dựng lời giải cho bài toán dự đoán `Depression`.

Một điểm tích cực khác là tài liệu và notebook đã phản ánh khá rõ tư duy thực nghiệm của nhóm. README mô tả tương đối đầy đủ mục tiêu bài toán, cách tổ chức quy trình chạy và nguồn dữ liệu; các notebook chính đều hiện diện với dung lượng lớn, cho thấy đã bao gồm cả phần mã nguồn, đầu ra và trực quan hóa. Đặc biệt, việc có thêm notebook nghiên cứu riêng cho các câu hỏi mở giúp tăng tính tái lập về mặt tư duy phân tích, không chỉ dừng lại ở việc huấn luyện mô hình như một "hộp đen".

Ở góc độ kỹ thuật, nhóm cũng đã thể hiện ý thức về tính cấu hình và khả năng tái sử dụng thông qua các biến thiết lập như `TARGET_COL = "Depression"` hay `DROP_COLS = ["id", "Name"]`. Bộ thư viện được lựa chọn tương đối phù hợp với bài toán dữ liệu bảng, bao gồm các mô hình cây, công cụ tối ưu siêu tham số và thư viện mã hóa biến phân loại; trong khi phần neural network được giữ ở dạng tùy chọn, thể hiện cách tiếp cận thực dụng và không phụ thuộc cứng vào một công nghệ duy nhất.

Từ toàn bộ quá trình đó, có thể kết luận rằng dự án đã xây dựng được một nền tảng phân tích và mô hình hóa khá đầy đủ cho bài toán dự đoán sức khỏe tinh thần. Kết quả quan trọng nhất không chỉ nằm ở hiệu năng của mô hình, mà còn ở chỗ nhóm đã xác định được một số tín hiệu có ý nghĩa như áp lực học tập/công việc, stress tài chính, chất lượng giấc ngủ và bối cảnh học tập – làm việc. Đây là cơ sở tốt để phát triển tiếp cả về mặt mô hình lẫn khả năng diễn giải trong các bước tiếp theo.

9.2 Hạn chế

Mặc dù có cấu trúc tốt, dự án hiện vẫn tồn tại một số hạn chế quan trọng liên quan đến dữ liệu, khả năng tái lập và mức độ sẵn sàng triển khai thực tế.

- **Hạn chế về dữ liệu và khả năng tái lập.** Dữ liệu thô `train/test` không được lưu trực tiếp trong repo mà phải tải thủ công từ Kaggle. Cách làm này phù hợp với ràng buộc bản quyền và dung lượng, nhưng hiện chưa có script hỗ trợ tải tự động bằng Kaggle API, cũng chưa có cơ chế versioning dữ liệu như DVC hay quy ước gắn nhãn phiên bản dữ liệu. Bên cạnh đó, `requirements.txt` mới chỉ mô tả các phụ thuộc ở mức tối thiểu, chưa khóa phiên bản cụ thể nên việc tái tạo đúng môi trường chạy giữa các máy vẫn còn hạn chế.

- **Hạn chế về tổ chức mã nguồn và tự động hóa.** Phần lớn logic hiện đang nằm trong notebook, thuận tiện cho quá trình khám phá nhưng chưa tối ưu cho tái sử dụng, kiểm thử và tích hợp liên tục. Dự án hiện cũng chưa có các script hoặc CLI riêng cho các bước chính như tiền xử lý, huấn luyện, đánh giá hay xuất mô hình. Ngoài ra, notebook `06_tuning_ensemble.ipynb` chưa thể hiện rõ mức độ hoàn thiện, làm cho phần tuning và ensemble chưa thật sự khép kín về mặt quy trình.

- **Hạn chế trong đánh giá mô hình.** Repo chưa có một tệp tổng hợp kết quả chuẩn hóa, chẳng hạn bảng so sánh `ROC-AUC`, `PR-AUC`, `F1-score` hay `Recall` giữa các mô hình. Dù trong notebook đã xuất hiện cấu hình `N_FOLDS = 5` và các thành phần như `StratifiedKFold`, mức độ nhất quán của quy trình *cross-validation* vẫn cần được thể hiện rõ hơn trong báo cáo. Bên cạnh đó, chưa thấy phân tích sâu về calibration, `PR-AUC`, lựa chọn threshold hay hành vi mô hình trong bối cảnh mất cân bằng lớp.

- **Hạn chế về kiểm soát rò rỉ dữ liệu và fairness.** Với dữ liệu sức khỏe tinh thần, việc kiểm tra *target leakage*, vai trò của biến định danh, hay sự khác biệt hiệu năng giữa các nhóm nhân khẩu học là rất quan trọng. Tuy nhiên, repo hiện chưa cho thấy một bước phân tích rò rỉ biến hoặc đánh giá fairness riêng biệt. Đây là khoảng trống đáng lưu ý nếu hệ thống được xem như một công cụ hỗ trợ ra quyết định.

- **Hạn chế về theo dõi thí nghiệm và lưu trữ mô hình.** Dự án chưa có cơ chế *experiment tracking* như MLflow hay Weights & Biases, cũng chưa thể hiện rõ cách lưu mô hình cùng metadata đi kèm như siêu tham số, metric, phiên bản dữ liệu hay thời điểm huấn luyện. Điều này khiến việc so sánh giữa các lần chạy và tái sử dụng mô hình về sau gặp nhiều khó khăn.

- **Hạn chế về tài liệu cộng tác và khía cạnh đạo đức.** README hiện khá hữu ích cho việc chạy notebook thủ công, nhưng vẫn thiếu các tài liệu hỗ trợ cộng tác như `CONTRIBUTING.md`, `LICENSE` hay `CODE_OF_CONDUCT`. Ngoài ra, vì đây là dữ liệu liên quan đến sức khỏe tâm thần, báo cáo cũng cần nhấn mạnh mạnh mẽ hơn rằng mô hình không thay thế chẩn đoán y khoa, đồng thời cần chú ý đến ẩn danh hóa, dữ liệu nhạy cảm và nguyên tắc sử dụng có trách nhiệm.

9.3 Hướng phát triển

Để nâng cấp dự án theo hướng vừa thực tiễn vừa có giá trị học thuật cao hơn, nhóm có thể triển khai roadmap theo ba mức ưu tiên sau.

9.3.1 Ưu tiên cao

- **Tự động hóa bước tải dữ liệu:** bổ sung script như `data/get_data.py` hoặc một target trong `Makefile` để tải dữ liệu từ Kaggle API, đồng thời hướng dẫn rõ cách cấu hình `kaggle.json`. Đây là bước quan trọng để giảm thao tác thủ công và giúp người khác tái lập dự án nhanh hơn.

- **Tạo môi trường reproducible:** thêm `environment.yml` hoặc một file khóa phụ thuộc như `requirements-lock.txt` với phiên bản Python và package cụ thể, từ đó giảm sai khác kết quả giữa các môi trường chạy.

- **Tách notebook thành module hoặc script:** di chuyển dần các phần tiền xử lý, tạo đặc trưng, huấn luyện và đánh giá sang các file như `src/preprocessing.py`, `src/models.py`, `src/train.py`. Khi đó, notebook có thể tập trung vào trình bày thí nghiệm, còn logic chính được chuẩn hóa và dễ kiểm thử hơn.

- **Theo dõi thí nghiệm và lưu mô hình:** tích hợp MLflow hoặc một công cụ tương đương để log siêu tham số, metric, artifact và mã định danh từng lần chạy; đồng thời lưu model bằng `joblib`, `pickle` hoặc định dạng phù hợp kèm metadata cần thiết.

9.3.2 Ưu tiên trung bình

- **Thiết lập CI cơ bản:** sử dụng GitHub Actions để chạy lint, kiểm tra định dạng mã và một số unit test nhỏ cho các hàm tiền xử lý hoặc huấn luyện cốt lõi.
- **Bổ sung kiểm tra rò rỉ dữ liệu và báo cáo phân phối:** viết các script hoặc test để kiểm tra biến định danh, *target leakage*, tỉ lệ missing values, cũng như sự khác biệt phân phối trước và sau khi chia tập dữ liệu.
- **Mở rộng đánh giá metric và calibration:** ngoài `ROC-AUC` và `F1-score`, nên theo dõi thêm `PR-AUC`, `Recall`, `Specificity`, `Brier Score` và calibration curve; nếu cần có thể áp dụng Platt scaling hoặc isotonic regression cho mô hình ensemble.
- **Xử lý mất cân bằng lớp tốt hơn:** thử nghiệm `class_weight`, `scale_pos_weight`, SMOTE hoặc focal loss để xem liệu có cải thiện khả năng phát hiện lớp thiểu số hay không.
- **Tăng cường explainability và tuning:** bổ sung SHAP hoặc TreeSHAP để giải thích mô hình, đồng thời sử dụng Optuna kết hợp *nested CV* hoặc *repeated stratified K-fold* để tối ưu siêu tham số một cách chặt chẽ hơn.

9.3.3 Ưu tiên thấp

- **Đóng gói và triển khai:** xây dựng `Dockerfile`, API suy luận đơn giản bằng FastAPI hoặc một giao diện demo để phục vụ việc trình bày và thử nghiệm mô hình ngoài notebook.
- **Theo dõi mô hình khi sử dụng thực tế:** nếu hệ thống được triển khai thử nghiệm, nên có cơ chế theo dõi *data drift* và cảnh báo khi phân phối dữ liệu đầu vào thay đổi đáng kể.
- **Versioning cho dữ liệu và mô hình:** áp dụng DVC hoặc một cơ chế manifest đơn giản để quản lý các phiên bản dữ liệu lớn, checkpoint mô hình và kết quả thí nghiệm.
- **Nâng cấp hệ sinh thái notebook:** có thể dùng Papermill để tham số hóa notebook hoặc một công cụ tương tự để tự động hóa việc chạy lại toàn bộ chuỗi thí nghiệm.

Tóm lại, hướng phát triển quan trọng nhất của dự án không chỉ là cải thiện thêm vài phần nghìn điểm số, mà là nâng cấp từ một bộ notebook nghiên cứu tốt thành một pipeline có thể tái lập, kiểm thử, giải thích và mở rộng một cách bài bản. Nếu thực hiện được các bước ưu tiên cao trước, dự án sẽ trở nên vững chắc hơn rất nhiều cả về giá trị học thuật lẫn giá trị kỹ thuật.

Tài liệu

[1] Kaggle, *Exploring Mental Health Data*, [Online]. Available: https://www.kaggle.com/.

[2] scikit-learn Developers, *scikit-learn: Machine Learning in Python*, [Online]. Available: https://scikit-learn.org/.

[3] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD, 2016.

[4] G. Ke, Q. Meng, T. Finley and others, *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, Advances in Neural Information Processing Systems (NeurIPS), 2017.

[5] L. Prokhorenkova, G. Gusev, A. Vorobev and others, *CatBoost: Unbiased Boosting with Categorical Features*, Advances in Neural Information Processing Systems (NeurIPS), 2018.

[6] T. Akiba, S. Sano, T. Yanase and others, *Optuna: A Next-Generation Hyperparameter Optimization Framework*, Proceedings of the 25th ACM SIGKDD, 2019.

[7] S. Lundberg and S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems (NeurIPS), 2017.